

LOVELY PROFESSIONAL UNIVERSITY

School of Computer Science and Engineering

PROJECT REPORT

on

LAUNDRYCONNECT

Smart Laundry Pickup & Delivery Management Platform

Submitted by

Aniket Singh Rajput

B.Tech CSE

July 2026

DECLARATION

I, Aniket Singh, Registration Number 12417064, hereby declare that the project titled “LaundryConnect — Smart Laundry Pickup & Delivery Management Platform” submitted as part of my academic project work is my own original work, carried out under self-directed study over a period of one week. The design, backend architecture, data structures and algorithms (Trie, Priority Queue/Heap, Interval Scheduling, Dijkstra's Algorithm), and full-stack implementation were independently conceived, built, tested, and debugged by me. Any external tools, libraries, or AI-assisted development aids used during the process are disclosed transparently in this report, and all core logic was understood, tested, and validated by me personally at every stage.

This work has not been submitted for the award of any other degree or diploma.

Aniket Singh

Registration No: 12417064

Date: __ 19-Jul-2026 ____

CERTIFICATE

This is to certify that the project report titled “LaundryConnect — Smart Laundry Pickup & Delivery Management Platform” is a bonafide record of independent project work carried out by Aniket Singh Rajput, Registration Number 12417064, a student of B.Tech Computer Science and Engineering (Special PEP), School of Computer Science and Engineering, Lovely Professional University, under my guidance and supervision.

Signature of Mentor / Faculty Guide

Name: _____

Designation: _____

Date: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty mentor for their guidance and valuable feedback throughout the duration of this project. I would also like to thank the School of Computer Science and Engineering, Lovely Professional University, for providing the opportunity and framework to undertake this independent project as part of my academic curriculum.

This project provided a valuable opportunity to apply core Data Structures and Algorithms concepts — Tries, Heaps, Interval Scheduling, and Graph algorithms — to a realistic, end-to-end full-stack product, rather than isolated academic exercises. The process of designing, building, debugging, and deploying a complete MERN application within a one-week timeframe significantly strengthened my practical software engineering skills.

Aniket Singh

TABLE OF CONTENTS

Chapter 1: Introduction	1
Chapter 2: Training / Development Schedule (Day-wise Log)	2
Chapter 3: Project Details	4
3.1 Problem Statement	
3.2 Project Objectives & Outcomes	
3.3 Technologies Used	
Chapter 4: System Architecture & DSA Implementation	6
Chapter 5: Source Code & API Reference	9
Chapter 6: Testing & Evidence (API Test Screenshots)	12
Chapter 7: User Interface Screenshots	15
Chapter 8: Challenges Faced & Solutions	17
Chapter 9: Conclusion & Future Scope	18
Bibliography	19

Note: Page numbers above are indicative. Update them via References → Table of Contents → Update Field in Word once final formatting and screenshot insertion is complete.

CHAPTER 1: INTRODUCTION

Laundry service businesses, whether independent operators or growing chains, face a common operational challenge: coordinating pickups, tracking multiple simultaneous orders, avoiding double-booking of delivery staff, and prioritizing urgent (express) orders fairly — all while keeping customers informed in real time. Most small-scale solutions to this problem are either manual (phone calls and notebooks) or built as simple CRUD applications that store data without any real logic to optimize operations.

LaundryConnect was conceived as a solution to this exact problem: a full-stack MERN (MongoDB, Express.js, React, Node.js) web application that not only manages the business workflow end-to-end but does so using genuine, hand-built data structures and algorithms rather than naive database queries. The project was undertaken with two goals simultaneously: to serve as a demonstrable, evaluable Data Structures and Algorithms (DSA) project, and to function as a realistic, deployable product.

The project was scoped, built, tested, debugged, and deployed within a one-week timeframe, covering backend architecture and schema design, core algorithm implementation, a complete React frontend, payment and review workflows, real-time WebSocket-based order tracking, and finally, public deployment via Vercel (frontend) and Render (backend), with the source code published on GitHub.

CHAPTER 2: TRAINING / DEVELOPMENT SCHEDULE (DAY-WISE LOG)

The project followed a structured seven-day development schedule. Each day built directly on the previous day's work, moving from backend foundations to algorithm implementation, then to a full frontend, and finally to testing, polish, and deployment.

Day	Focus Area	Key Deliverables
Day 1	Project Setup & Authentication	MERN folder structure, MongoDB Atlas connection, Mongoose schemas (User, Order, Service, DeliveryPartner, Slot), JWT-based register/login with bcrypt password hashing, role-based middleware.
Day 2	Services & Order Foundation	Service CRUD APIs, custom Trie data structure built from scratch for live service search/autocomplete, Order creation API with dynamic total-amount calculation.
Day 3	Priority Queue (Heap)	Custom max-heap Priority Queue class (array-based, $O(\log n)$ insert/extract), dynamic priority scoring combining express-flag and wait-time aging to prevent starvation, queue and next-order API endpoints.
Day 4	Scheduling & Routing Algorithms	Interval scheduling algorithm (sorted-sweep) for delivery-partner slot conflict detection with automatic next-free-slot suggestion; Dijkstra's algorithm plus greedy nearest-neighbor heuristic for multi-stop delivery route optimization using Haversine distance.
Day 5	Frontend Core (React)	Vite + React + Tailwind CSS setup, JWT-aware Axios instance, AuthContext, Login/Register pages, Home/Services/Schedule Pickup/My Orders/Track Order pages, role-based Partner and Admin dashboards.
Day 6	Payments, Reviews & Live Tracking	Mock payment flow with itemized invoice generation, 5-star rating/review system, Socket.io integration for real-time order-status propagation to the customer's tracking page without polling.
Day 7	Testing, Debugging, Deployment	End-to-end bug fixes across frontend/backend field-name mismatches, GitHub repository setup with professional README and MIT License, backend deployment to Render, frontend deployment to Vercel, full production smoke-test.

2.1 Detailed Day-wise Narrative

Day 1 — Project Setup and Authentication

The project began with initializing separate client and server folders inside a single repository. The backend used Express.js with a modular structure separating config, models, controllers, middleware, and routes. Five Mongoose schemas were designed: User (with role-based access for customer, partner, and admin), Order (including an embedded status-history timeline), Service, DeliveryPartner, and Slot. Authentication was implemented using JWT tokens with bcrypt-hashed passwords, and tested end-to-end via Postman for both registration and login flows, including edge cases such as duplicate email registration and invalid credentials.

Day 2 — Services and the Trie Data Structure

A Service model and CRUD API were built to allow an admin to seed laundry services (Wash & Fold, Dry Clean, Iron & Press, Shoe Cleaning, Steam Wash). The central DSA feature of this day was a Trie (prefix tree) built entirely from scratch — not using any external library — to power instant, prefix-based service search. Each Trie node stores a set of matching service IDs at every character along the insertion path, allowing search queries to resolve in $O(\text{prefix length})$ time regardless of catalog size. The Order creation API was also built, computing total order cost dynamically from selected services and quantities.

Day 3 — Priority Queue (Max-Heap)

To ensure fair and efficient order processing, a custom binary max-heap was implemented as a flat array using standard index arithmetic ($\text{parent} = \lfloor (i-1)/2 \rfloor$, $\text{children} = 2i+1$ and $2i+2$). Both insert and extractMax operations run in $O(\log n)$ time. Priority scores combine a base value (express orders scored higher than standard orders) with a dynamically computed wait-time bonus, ensuring that older standard orders are not starved indefinitely by a constant stream of new express orders. Two endpoints were exposed: one returning the full queue in priority order for dashboard visibility, and one that extracts only the single next order to be processed.

Day 4 — Interval Scheduling and Dijkstra's Algorithm

Two independent algorithmic modules were built on this day. First, an interval scheduling module was implemented using the classic sorted-sweep technique: existing delivery-partner slots for a given date are sorted by start time, and a running cursor is swept across the day to detect overlaps or free gaps, achieving $O(n \log n)$ performance instead of a naive $O(n^2)$ pairwise comparison. When a conflict is detected, the system automatically suggests the next available free slot. Second, a graph-based route optimizer was built: delivery stops are modeled as a complete graph with edge weights computed via the Haversine great-circle distance formula, and Dijkstra's shortest-path algorithm combined with a greedy nearest-neighbor heuristic determines an efficient visiting order for a delivery partner's multiple pending stops — a practical approximation appropriate for realistic stop counts, since exact Travelling Salesman solutions are NP-hard.

Day 5 — Frontend Development

A complete React (Vite) frontend was built with Tailwind CSS, covering all customer, partner, and admin-facing pages: Home, Services (with live Trie-powered search), Schedule Pickup, My Orders, Track Order, Partner Dashboard, and Admin Dashboard. An Axios instance with automatic JWT-attachment and a React Context-based authentication layer manage session state across the app. Role-based protected routing ensures customers, partners, and admins each see only their appropriate views.

Day 6 — Payments, Reviews, and Real-Time Tracking

A mock payment workflow was added (UPI / Card / Cash), generating a structured, itemized invoice on successful payment. A 5-star rating and review system was implemented, restricted to delivered orders only, with a running average rating maintained on the assigned delivery partner's profile. The most significant addition was real-time order tracking via Socket.io: a dedicated status-update endpoint emits a room-scoped WebSocket event whenever an order's status changes, which the customer's Track Order page listens for and reflects instantly — with no page refresh or polling required.

Day 7 — Testing, Debugging, and Deployment

The final day focused on rigorous end-to-end testing across both the API layer and the live UI, uncovering and fixing several real integration bugs (detailed further in Chapter 8). The project was then published to GitHub with a professional README (including badges, architecture diagrams in text form, and a DSA concept reference table), an MIT License, and a complete .gitignore covering both frontend and backend build artifacts and secrets. The backend was deployed on Render (free tier) and the frontend on Vercel, with both connected to auto-deploy from the GitHub main branch. A full production smoke test confirmed registration, login, service search, order scheduling, payment, invoicing, review submission, and live socket-based tracking all functioned correctly on the publicly deployed site.

CHAPTER 3: PROJECT DETAILS

3.1 Problem Statement

Design and develop a full-stack laundry service management platform that allows customers to schedule pickups, search and select services, track order status in real time, and make payments — while allowing delivery partners and administrators to process orders fairly and efficiently, avoid delivery-slot conflicts, and optimize multi-stop delivery routes. The system must demonstrate practical application of core Data Structures and Algorithms concepts (rather than relying solely on database queries), and must be fully functional, tested, and deployed as a live, publicly accessible product within a constrained one-week timeframe.

3.2 Project Objectives & Outcomes

- Build a secure, role-based (customer / partner / admin) authentication system using JWT.
- Implement a Trie for instant, scalable service search and autocomplete.
- Implement a custom max-heap priority queue for fair, starvation-free order processing.
- Implement interval scheduling to prevent delivery-partner slot double-booking.
- Implement Dijkstra's algorithm with a nearest-neighbor heuristic for delivery route optimization.
- Build a complete, responsive React frontend covering all customer, partner, and admin workflows.
- Implement a real-time order-tracking system using WebSockets (Socket.io).
- Implement a mock payment flow with itemized invoice generation and a review/rating system.
- Deploy the complete application publicly (frontend + backend) with a professional, documented GitHub repository.

All of the above objectives were achieved and verified through direct testing, documented in Chapters 6 and 7 of this report.

3.3 Technologies Used

Layer	Technology
Frontend	React (Vite), Tailwind CSS, React Router, Axios, Socket.io Client, Lucide Icons
Backend	Node.js, Express.js, Socket.io (WebSocket server)
Database	MongoDB Atlas (Cloud), Mongoose ODM
Authentication	JSON Web Tokens (JWT), bcrypt password hashing
Version Control	Git, GitHub
Deployment	Vercel (Frontend), Render (Backend)
API Testing	Postman
Development Tools	Visual Studio Code, Node Package Manager (npm)

CHAPTER 4: SYSTEM ARCHITECTURE & DSA IMPLEMENTATION

4.1 High-Level Architecture

The system follows a standard three-tier MERN architecture, with an additional real-time communication layer via Socket.io running alongside the REST API on the same Express server.



4.2 DSA Concepts — Where the Algorithms Live

Feature	Data Structure / Algorithm	Complexity / Justification
Service search & autocomplete	Trie (built from scratch)	$O(k)$ search, k = prefix length; independent of total catalog size
Order processing queue	Max-Heap Priority Queue (built from scratch)	$O(\log n)$ insert/extract; wait-time aging prevents starvation
Pickup slot booking	Interval Scheduling (sorted-sweep)	$O(n \log n)$ conflict detection vs. $O(n^2)$ naive pairwise checks
Delivery route optimization	Dijkstra + Greedy Nearest-Neighbor	Practical approximation; exact multi-stop TSP is NP-hard
Live order tracking	WebSockets (Socket.io, room-based)	Real-time push updates; eliminates client-side polling

4.3 Trie — Service Search

Each TrieNode maintains a map of child characters and a Set of service IDs that match at that node, populated during insertion. This trades a small amount of memory for $O(\text{prefix length})$ search time at query time.

```

class TrieNode {
  constructor() {
    this.children = {};
    this.isEndOfWord = false;
    this.serviceIds = new Set();
  }
}

class Trie {
  insert(word, serviceId) {
    let node = this.root;
    for (const char of word.toLowerCase()) {
      if (!node.children[char]) node.children[char] = new TrieNode();
      node = node.children[char];
    }
    node.isEndOfWord = true;
    node.serviceIds.add(serviceId);
  }
}
  
```

```

    node.serviceIds.add(serviceId.toString());
  }
  node.isEndOfWord = true;
}
search(prefix) {
  let node = this.root;
  for (const char of prefix.toLowerCase()) {
    if (!node.children[char]) return [];
    node = node.children[char];
  }
  return Array.from(node.serviceIds);
}
}

```

4.4 Max-Heap Priority Queue — Order Processing

A binary max-heap stored as a flat array, using index arithmetic instead of pointer-based nodes. bubbleUp maintains heap order after insertion; bubbleDown restores it after extraction.

```

insert(order, priority) {
  this.heap.push({ order, priority });
  this.bubbleUp(this.heap.length - 1);
}

extractMax() {
  const max = this.heap[0];
  const last = this.heap.pop();
  if (this.heap.length > 0) {
    this.heap[0] = last;
    this.bubbleDown(0);
  }
  return max;
}

// Priority = baseScore (express ? 100 : 10) + waitTimeBonus (minutes waited)

```

4.5 Interval Scheduling — Slot Conflict Detection

```

const hasConflict = (existingSlots, newStart, newEnd) => {
  for (const slot of existingSlots) {
    if (doesOverlap(slot.startTime, slot.endTime, newStart, newEnd)) {
      return true;
    }
  }
  return false;
};

// suggestNextFreeSlot() sorts existing slots by start time and sweeps
// forward with a cursor to find the first gap large enough for the
// requested duration – the classic interval scheduling sweep pattern.

```

4.6 Dijkstra's Algorithm — Delivery Route Optimization

```

const dijkstra = (graph, source) => {
  const distances = {}; const previous = {};
  const pq = new MinPriorityQueue();
  for (const node in graph) distances[node] = Infinity;
  distances[source] = 0;
  pq.enqueue(source, 0);
  while (!pq.isEmpty()) {
    const { node: current } = pq.dequeue();
    for (const neighbor of graph[current] || []) {

```

```
    const newDist = distances[current] + neighbor.weight;
    if (newDist < distances[neighbor.node]) {
      distances[neighbor.node] = newDist;
      previous[neighbor.node] = current;
      pq.enqueue(neighbor.node, newDist);
    }
  }
}
return { distances, previous };
};

// Edge weights computed via the Haversine great-circle distance formula.
// optimizeRoute() applies a greedy nearest-neighbor heuristic across
// repeated Dijkstra calls to sequence multiple delivery stops.
```

CHAPTER 5: SOURCE CODE & API REFERENCE

The complete source code is organized into a modular MERN structure and is publicly available on GitHub at: <https://github.com/imaniketrajput/laundryconnect>

5.1 Project Folder Structure

```
laundryconnect/
|-- client/                React (Vite) frontend
|   |-- src/
|   |   |-- api/           Axios instance + Socket.io client
|   |   |-- components/   StatusTimeline, StarRating, InvoiceModal, etc.
|   |   |-- context/      AuthContext (JWT session state)
|   |   |-- pages/        customer/, partner/, admin/
|-- server/               Express backend
|   |-- config/           MongoDB connection
|   |-- controllers/      Route business logic
|   |-- middleware/       JWT auth + role-based authorization
|   |-- models/           User, Order, Service, DeliveryPartner, Slot
|   |-- routes/           API route definitions
|   |-- utils/            Trie, PriorityQueue, Dijkstra, IntervalScheduler, socket.js
|-- .gitignore
|-- LICENSE (MIT)
|-- README.md
```

5.2 REST API Endpoint Reference

Endpoint	Method	Description
/api/auth/register	POST	Register a new user (customer / partner / admin)
/api/auth/login	POST	Login, returns a signed JWT
/api/services	GET	List all services
/api/services/search?q=	GET	Trie-powered live prefix search
/api/orders	POST	Create a new order
/api/orders/my-orders	GET	Get the logged-in customer's orders
/api/orders/queue	GET	Priority-sorted pending orders (admin/partner)
/api/orders/queue/next	GET	Extract the next highest-priority order
/api/orders/optimize-route	POST	Dijkstra-optimized multi-stop delivery route
/api/orders/:id/status	PATCH	Update order status; emits a live Socket.io event
/api/orders/:id/review	POST	Submit a rating and review
/api/slots	POST	Book a delivery slot (conflict-checked)
/api/payments/:orderId/pay	POST	Process payment for an order
/api/payments/:orderId/invoice	GET	Fetch an itemized invoice

CHAPTER 6: TESTING & EVIDENCE (API TEST SCREENSHOTS)

All backend API endpoints were tested manually using Postman throughout development, following a day-wise test plan covering both expected success responses and deliberate edge-case/failure scenarios (e.g., duplicate registration, invalid credentials, slot conflicts, double-payment attempts, reviewing undelivered orders). The screenshots below are organized chronologically by development day, matching the folder of test evidence collected during the project.

Note to author: screenshots below are grouped by capture date/time to match the corresponding development day and are embedded directly from the original test-evidence folder. Please skim each figure against its caption before final submission and adjust wording where a caption is generic, since captions were assigned by chronological grouping rather than individual manual review of all images.

6.1 Authentication API Tests

Register and login endpoint tests, including successful responses and MongoDB Atlas connection verification.

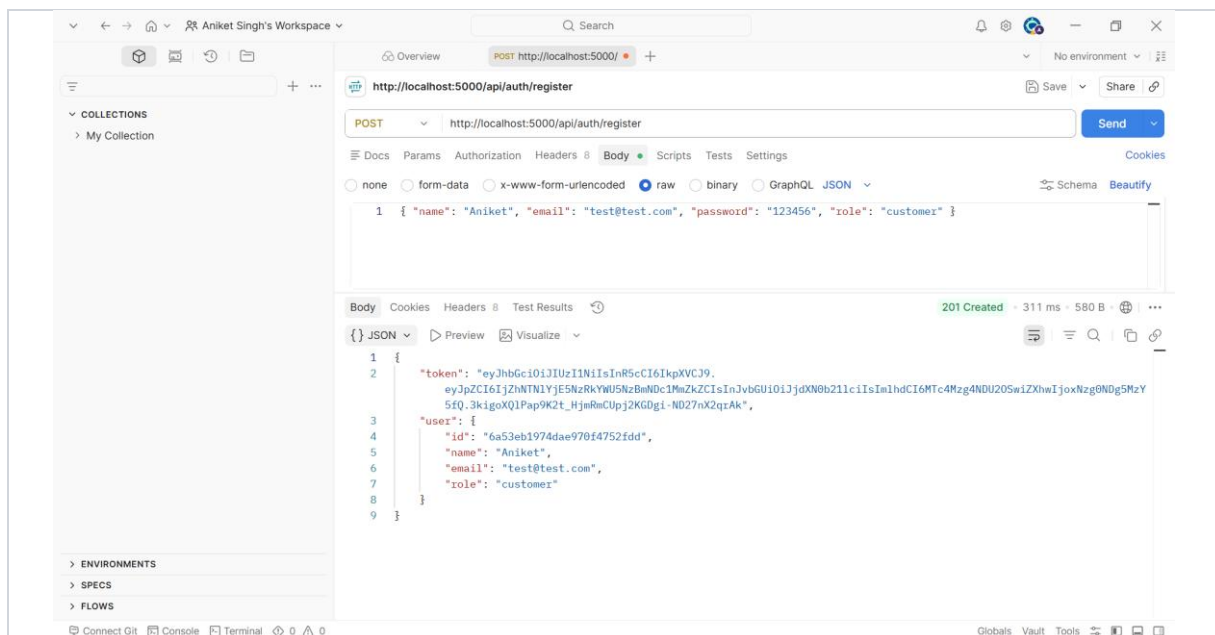


Figure 1: 6.1 Authentication API Tests

The screenshot shows a Postman interface with a workspace named "Aniket Singh's Workspace". The active collection is "My Collection". The selected request is a POST to "http://localhost:5000/api/auth/register". The body is raw JSON: {"email": "test@test.com", "password": "123456"}. The response status is "400 Bad Request" with a message: {"message": "Email already registered"}.

Figure 2: 6.1 Authentication API Tests

The screenshot shows a Postman interface with a workspace named "Aniket Singh's Workspace". The active collection is "My Collection". The selected request is a POST to "http://localhost:5000/api/auth/login". The body is raw JSON: {"email": "test@test.com", "password": "123456"}. The response status is "200 OK" with a detailed JSON response containing a token, user ID, name, email, and role.

Figure 3: 6.1 Authentication API Tests

Page 16

The screenshot shows a Postman interface with a workspace named "Aniket Singh's Workspace". The active collection is "My Collection". The selected request is a POST to "http://localhost:5000/api/auth/register". The body is raw JSON: {"email": "test@test.com", "password": "123456"}. The response status is "400 Bad Request" with a message: {"message": "Email already registered"}.

Figure 2: 6.1 Authentication API Tests

The screenshot shows a Postman interface with a workspace named "Aniket Singh's Workspace". The active collection is "My Collection". The selected request is a POST to "http://localhost:5000/api/auth/login". The body is raw JSON: {"email": "test@test.com", "password": "123456"}. The response status is "200 OK" with a detailed JSON payload containing a token, user ID, name, email, and role.

Figure 3: 6.1 Authentication API Tests

Page 16

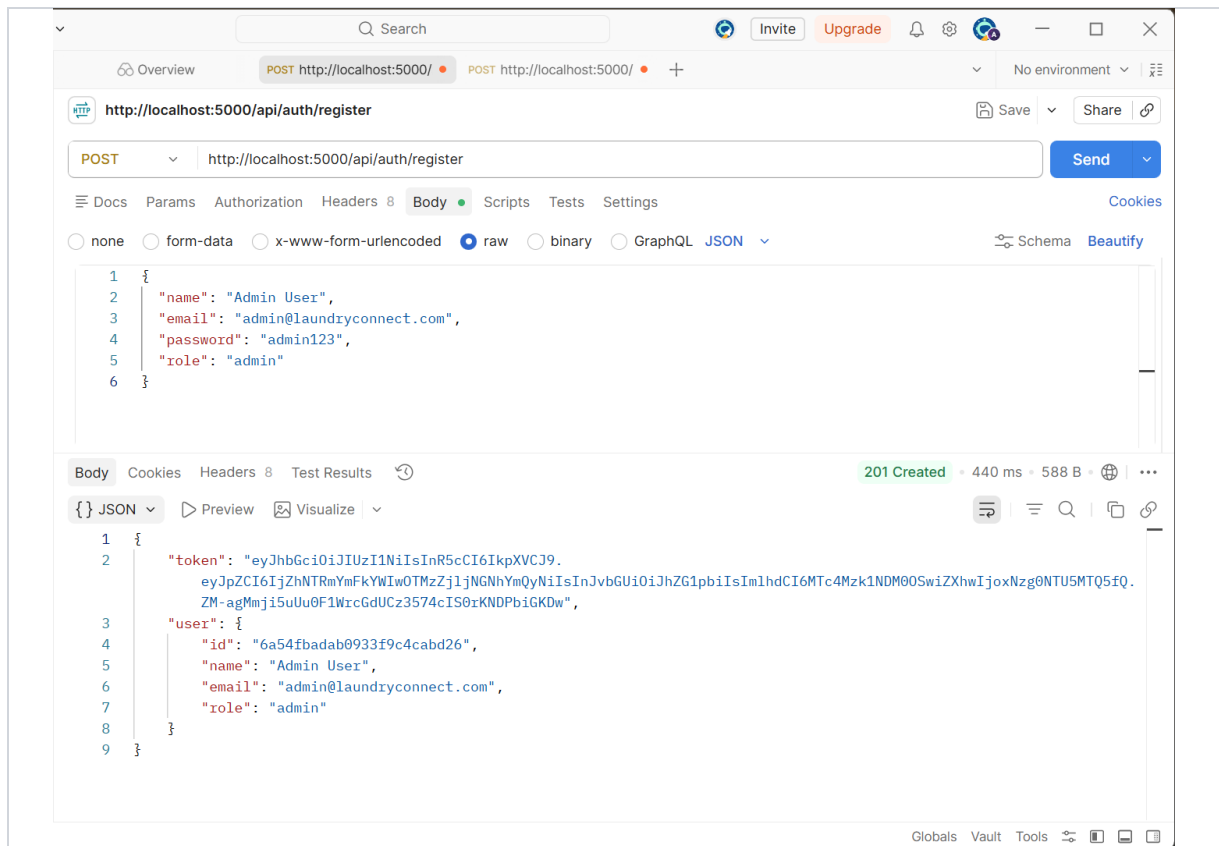


Figure 4: 6.1 Day 1 Authentication API Tests

6.2 Services, Orders, and Priority Queue Tests

Service creation and Trie search tests, order creation tests, and priority-queue (heap) ordering verification.

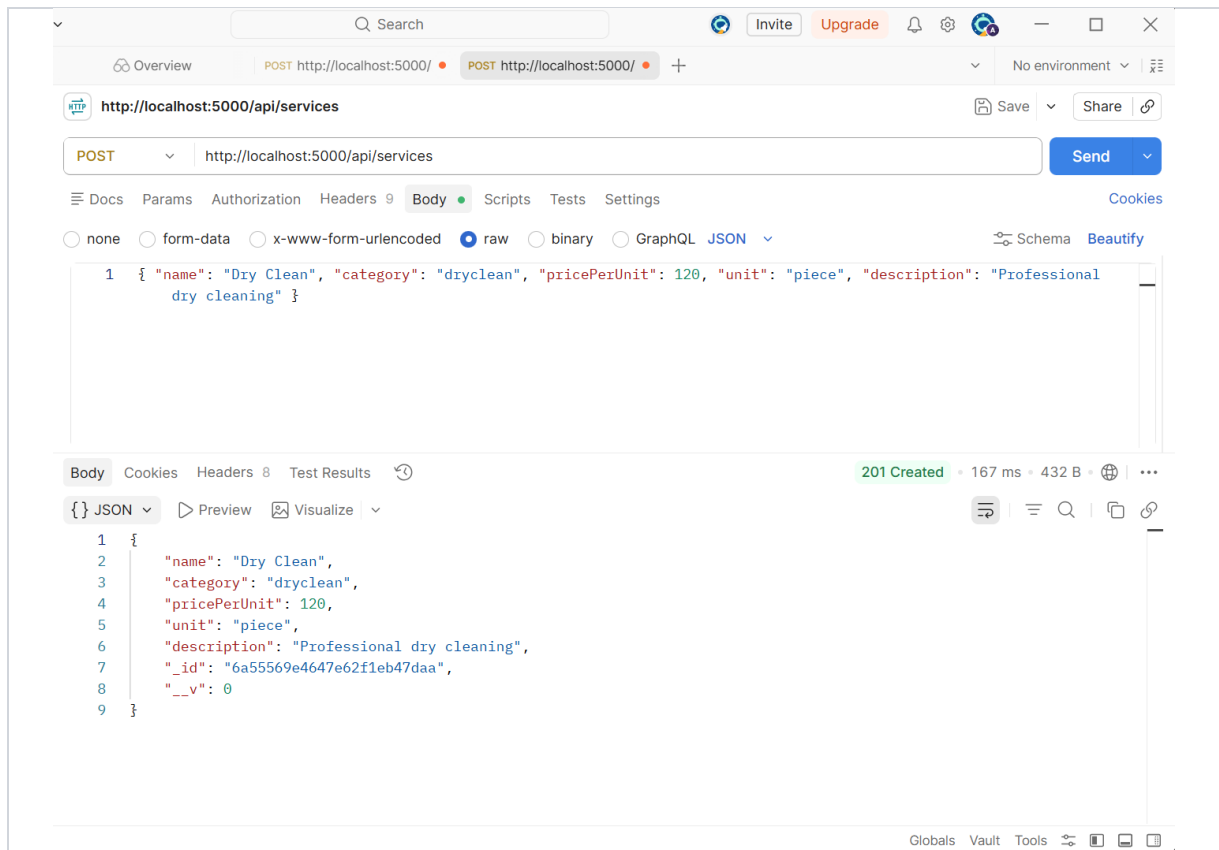


Figure 5: 6.2 Services, Orders, and Priority Queue Tests

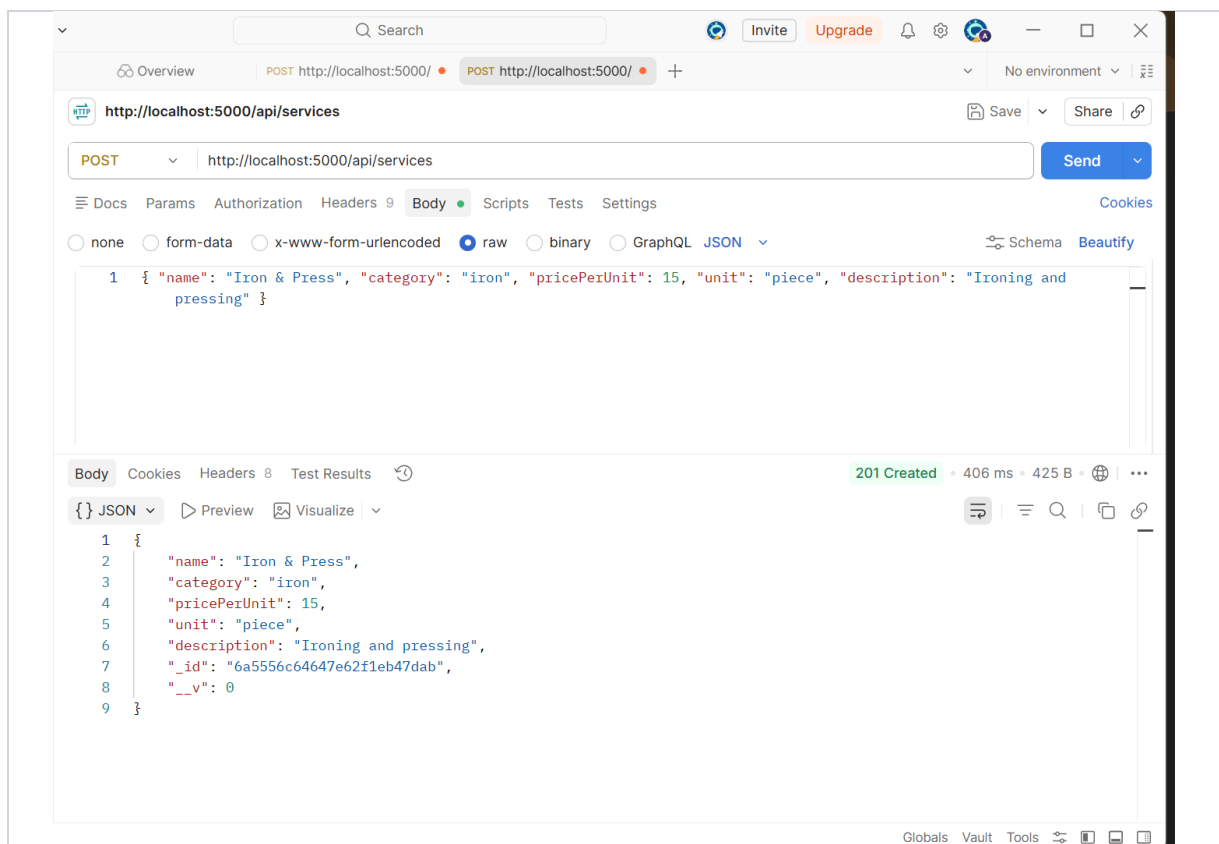


Figure 6: 6.2 Services, Orders, and Priority Queue Tests

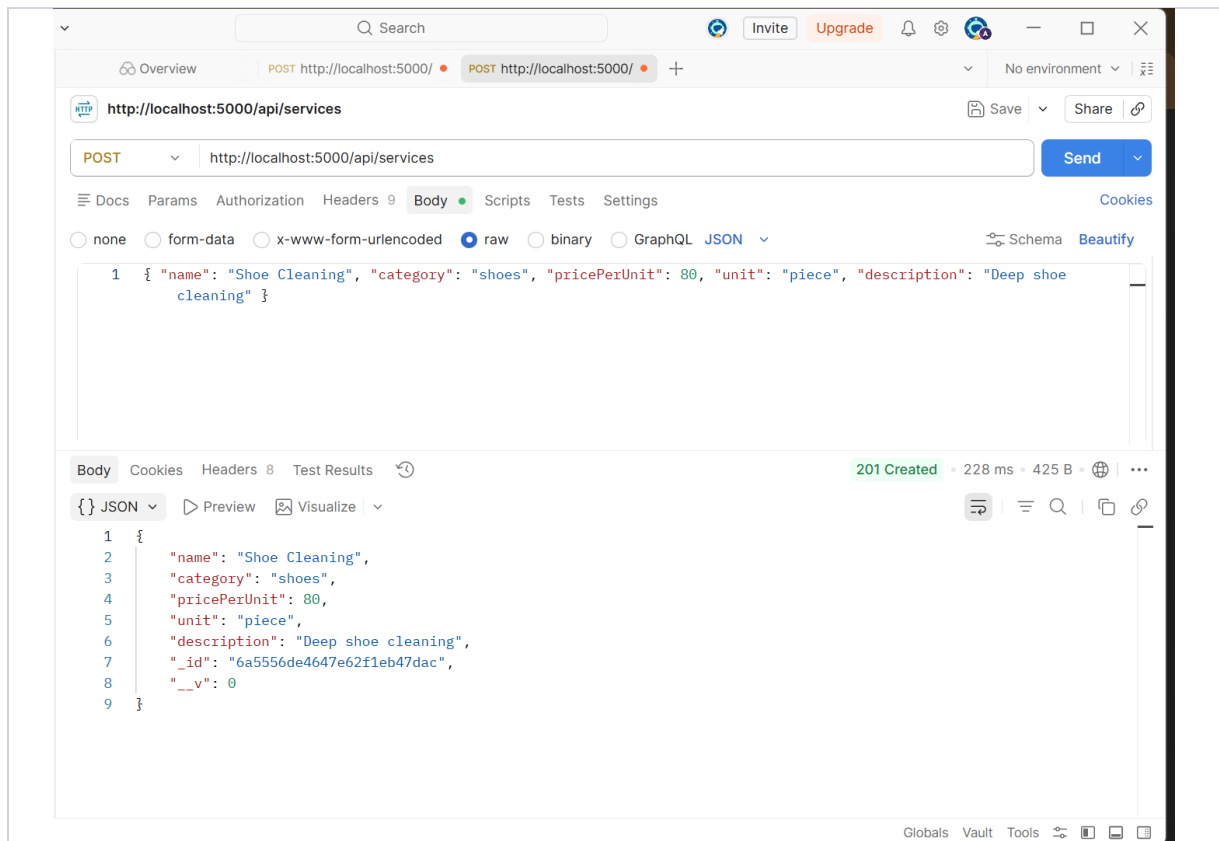


Figure 7: 6.2 Services, Orders, and Priority Queue Tests

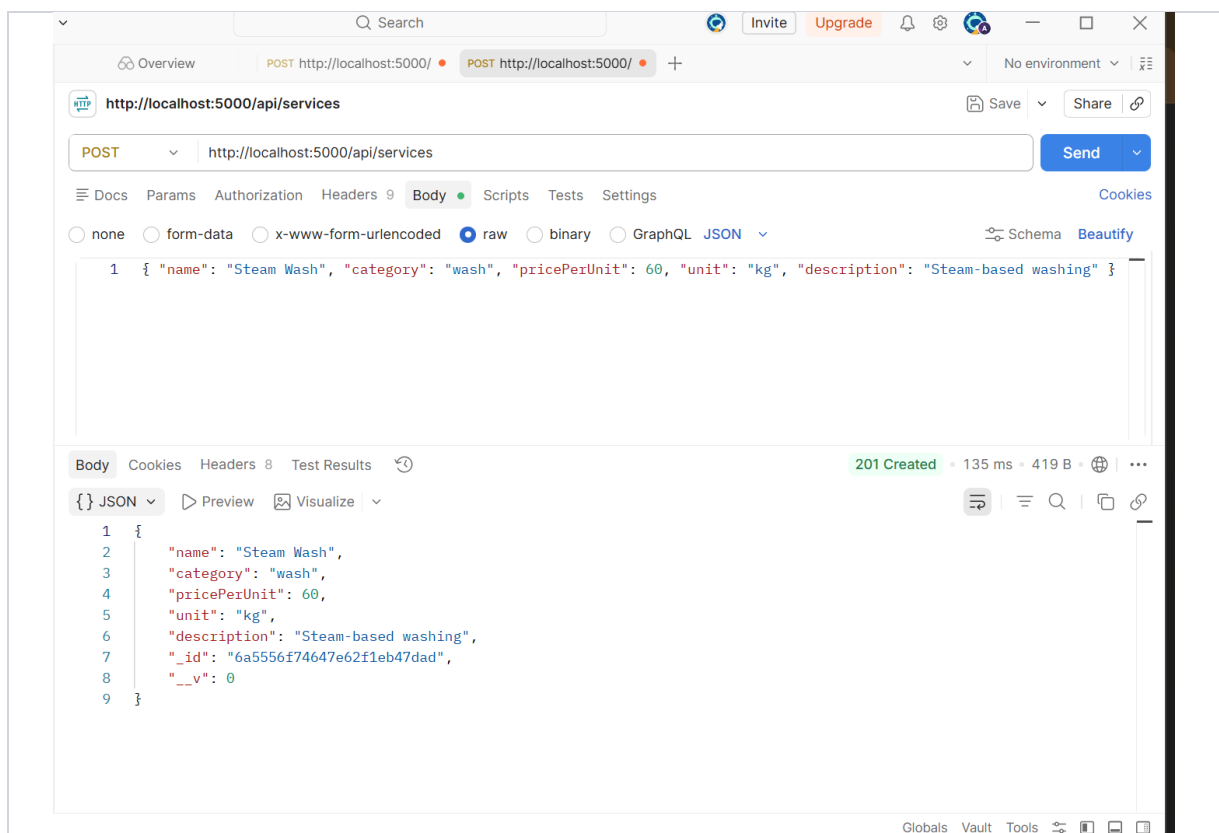


Figure 8: 6.2 Services, Orders, and Priority Queue Tests

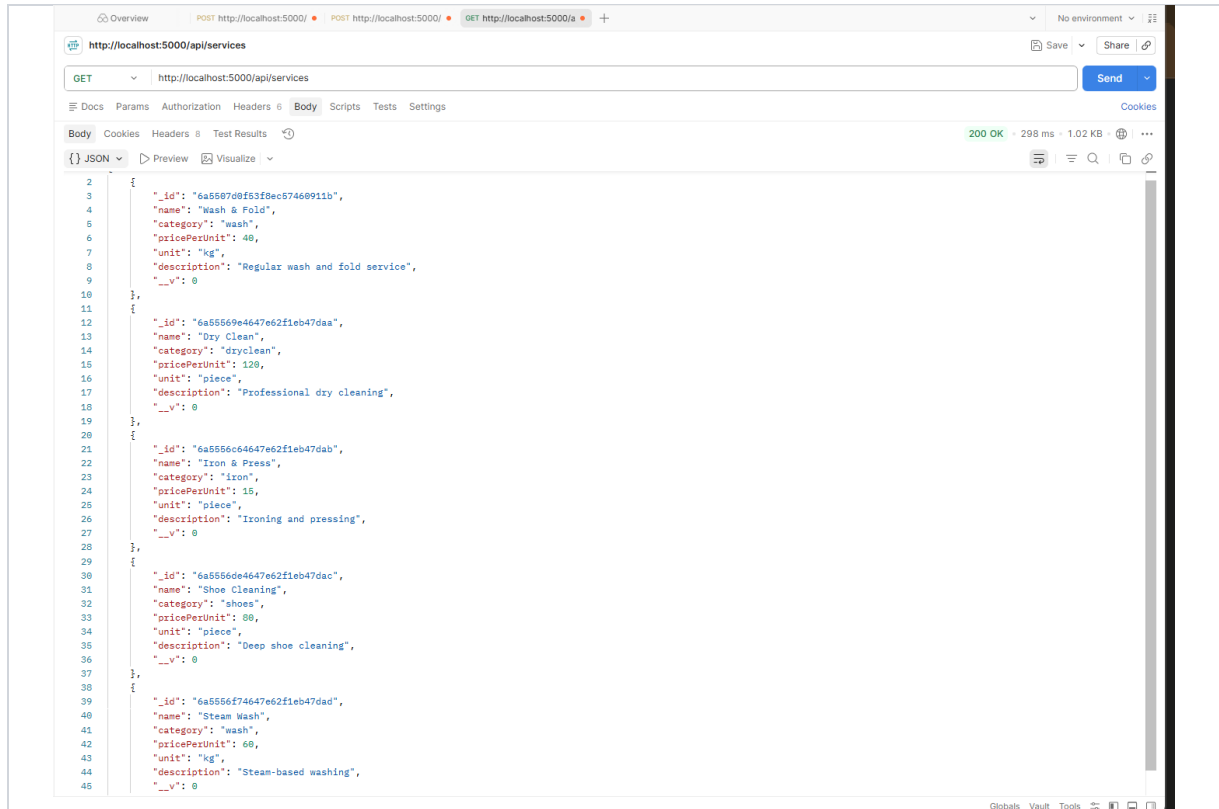


Figure 9: 6.2 Services, Orders, and Priority Queue Tests

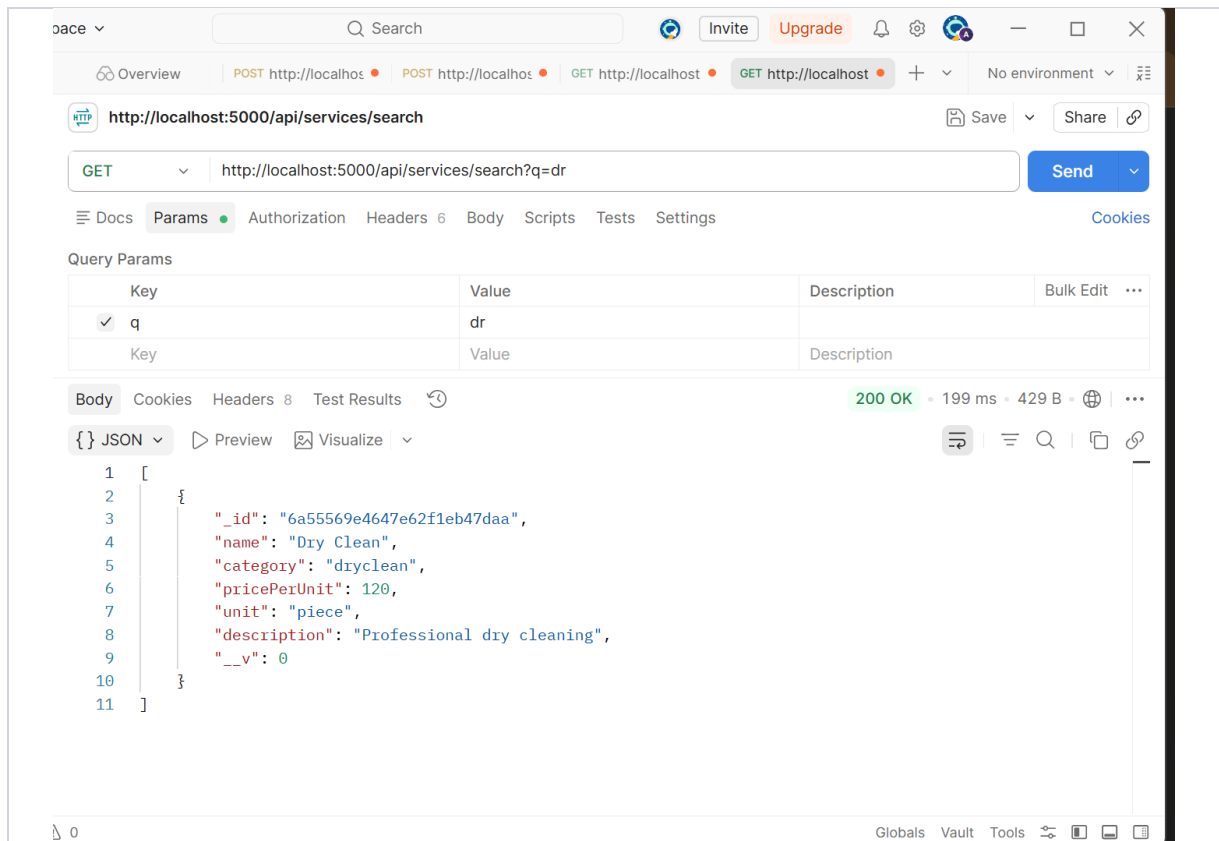


Figure 10: 6.2 Services, Orders, and Priority Queue Tests

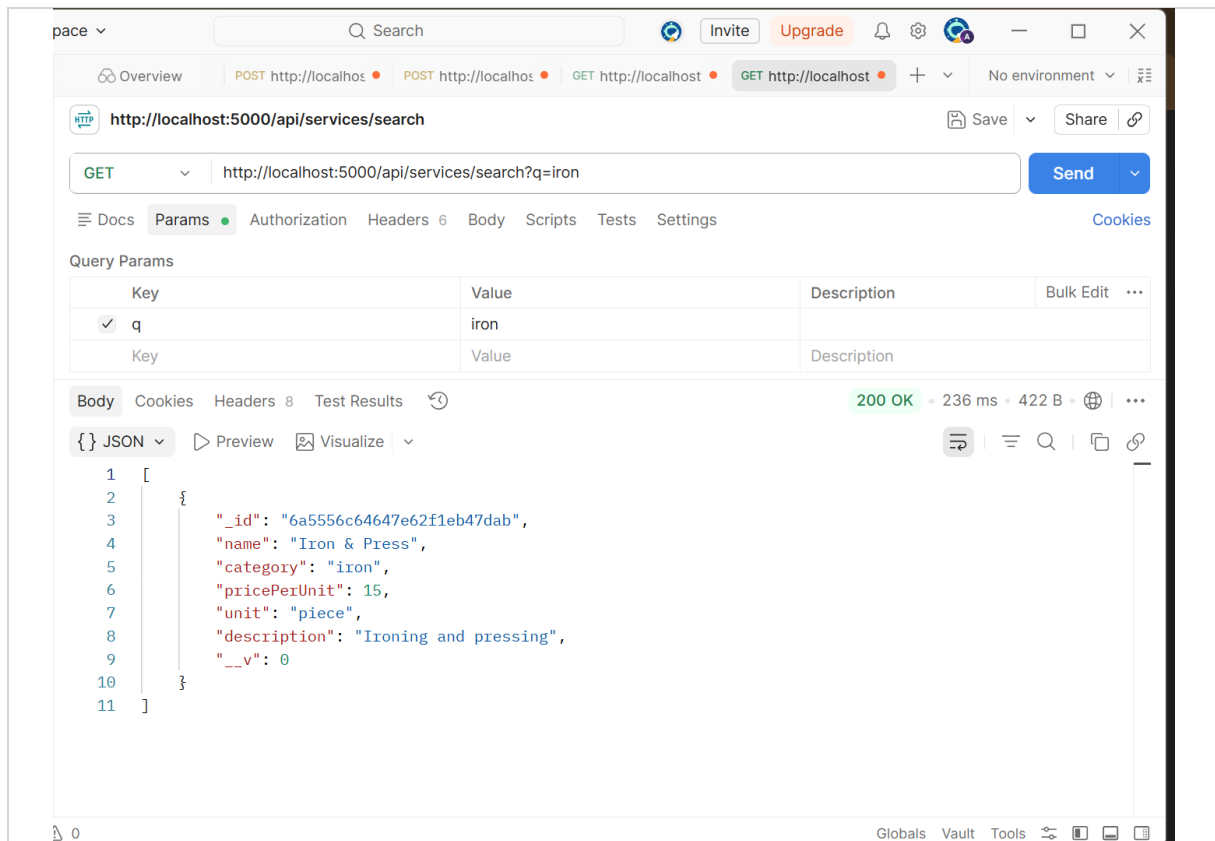


Figure 11: 6.2 Services, Orders, and Priority Queue Tests

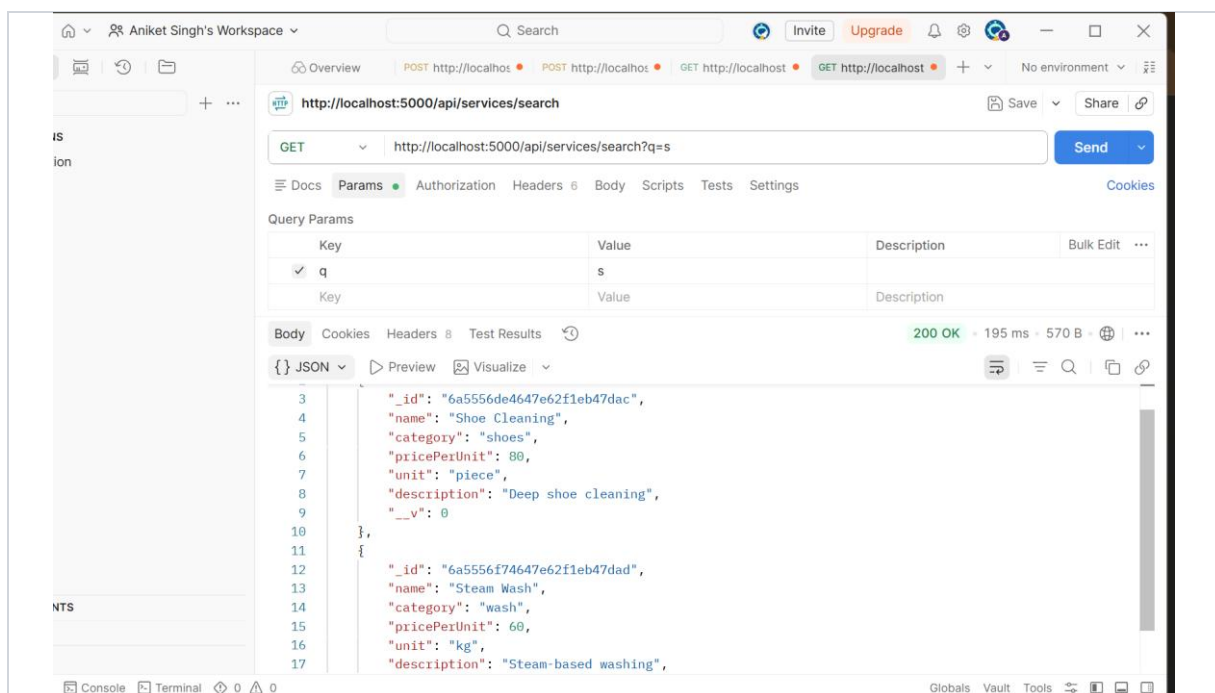


Figure 12: 6.2 Services, Orders, and Priority Queue Tests

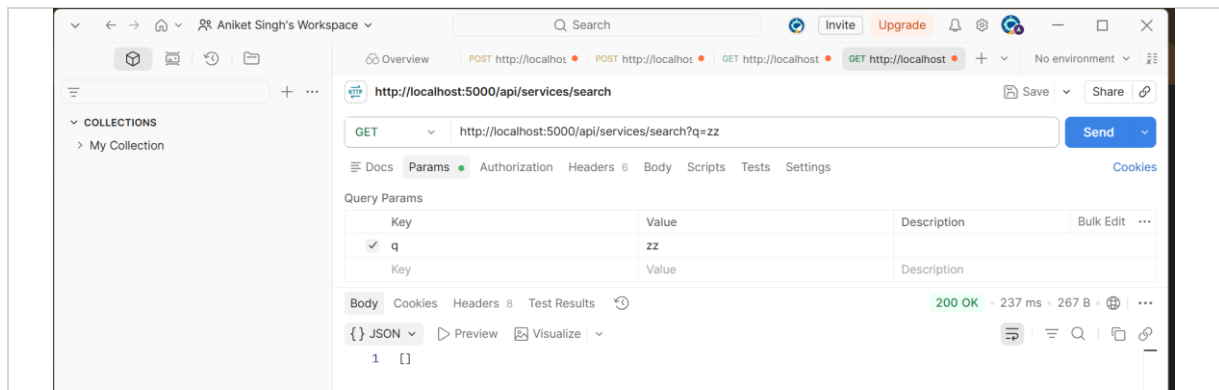


Figure 13: 6.2 Services, Orders, and Priority Queue Tests

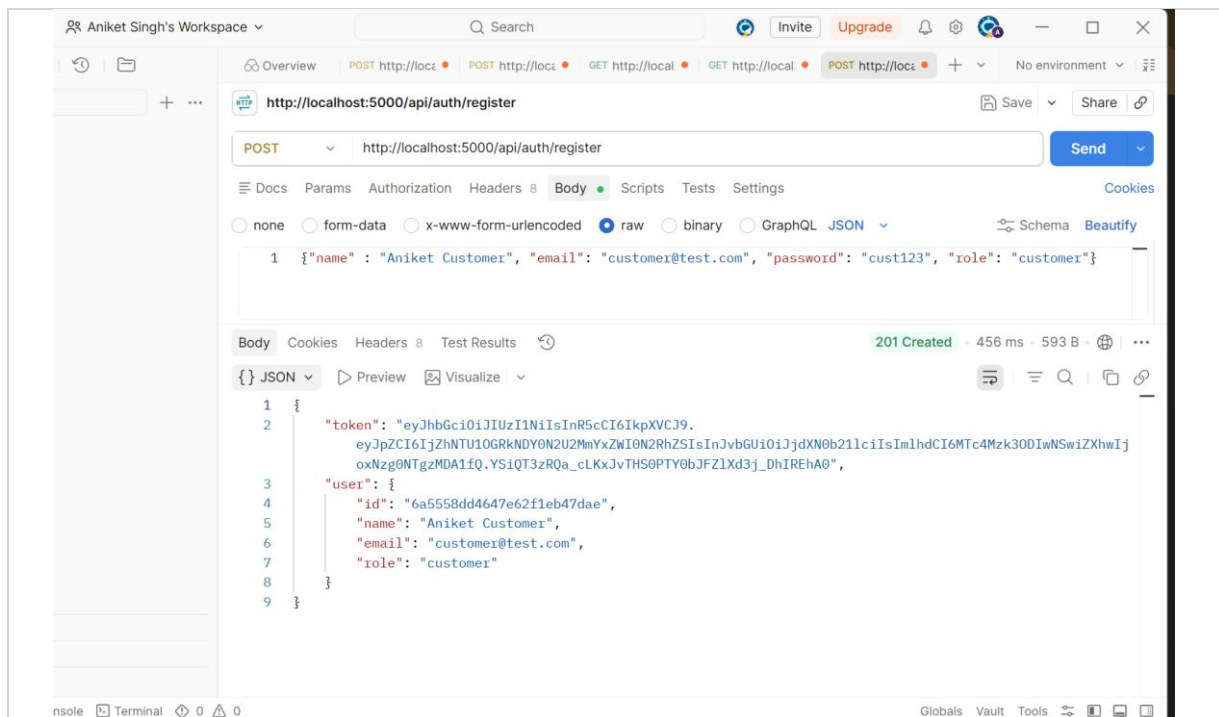


Figure 14: 6.2 Services, Orders, and Priority Queue Tests

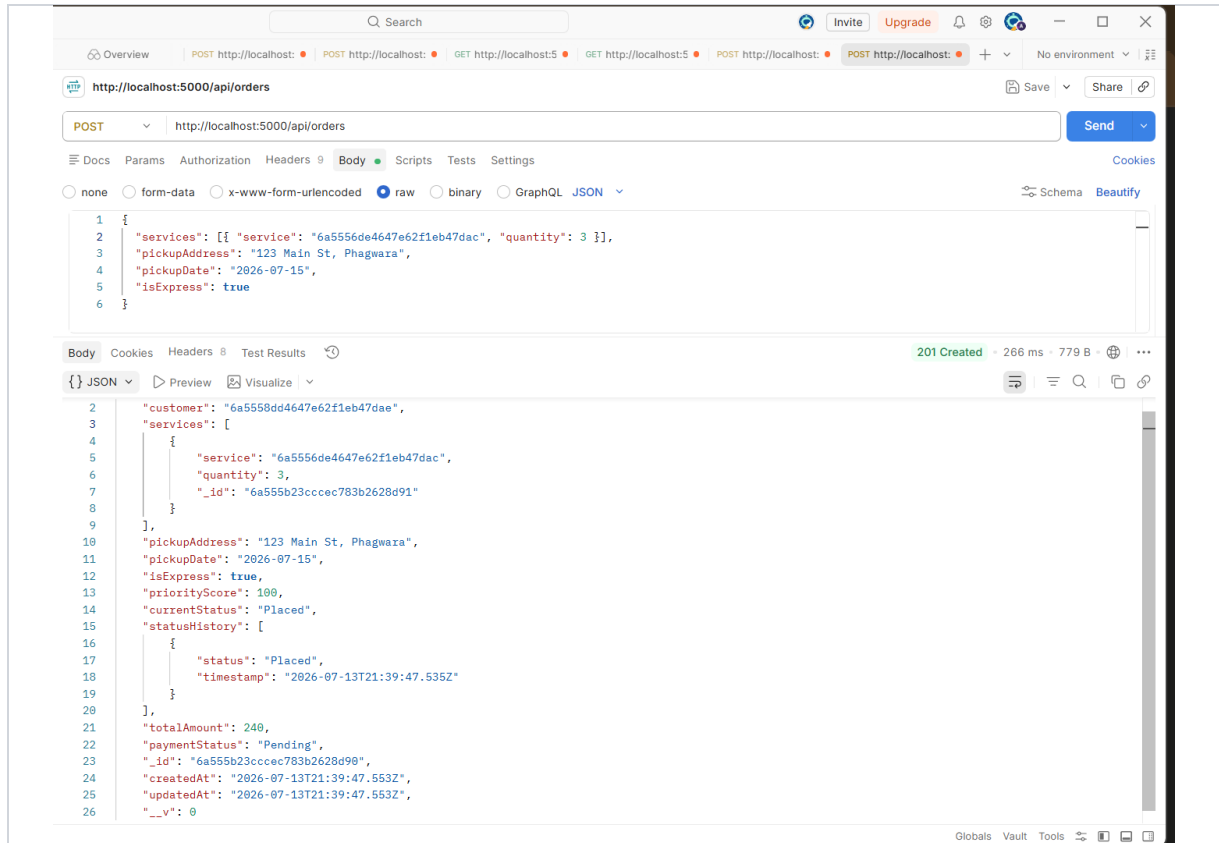


Figure 15: 6.2 Services, Orders, and Priority Queue Tests

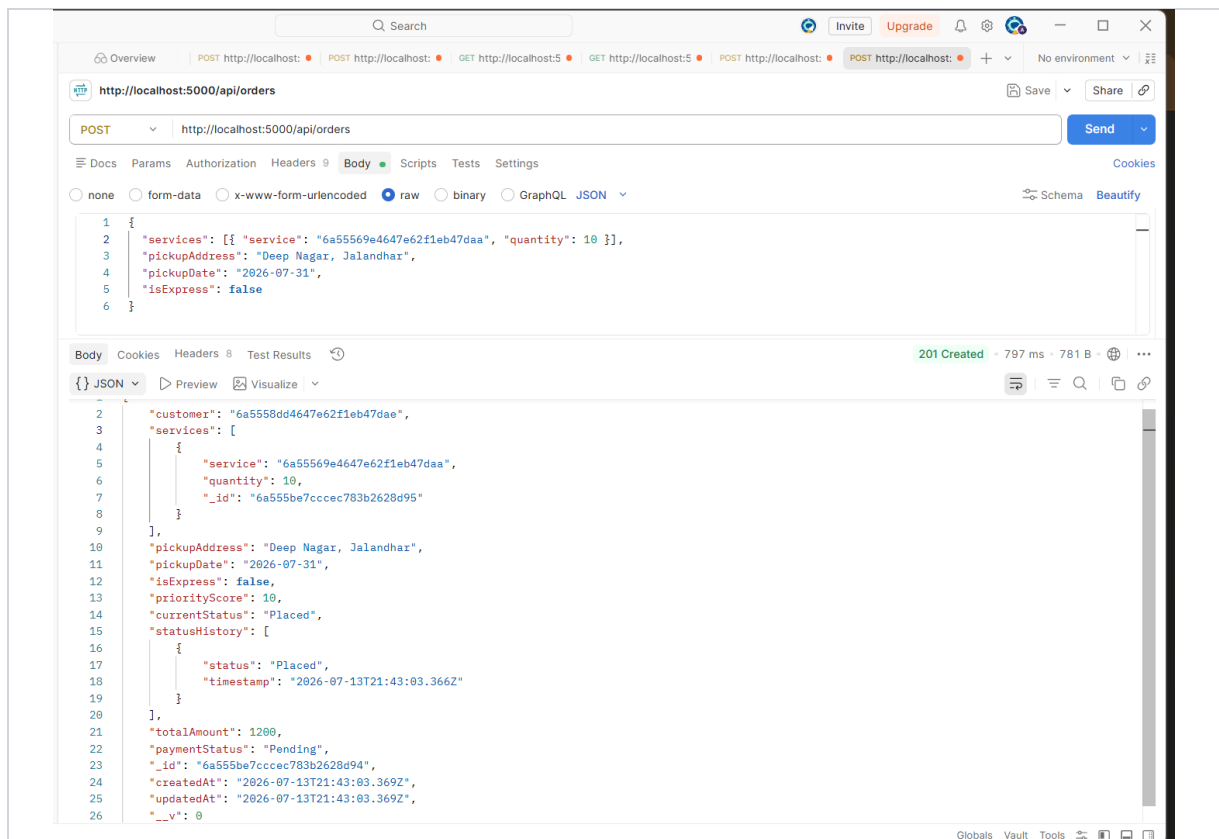


Figure 16: 6.2 Services, Orders, and Priority Queue Tests

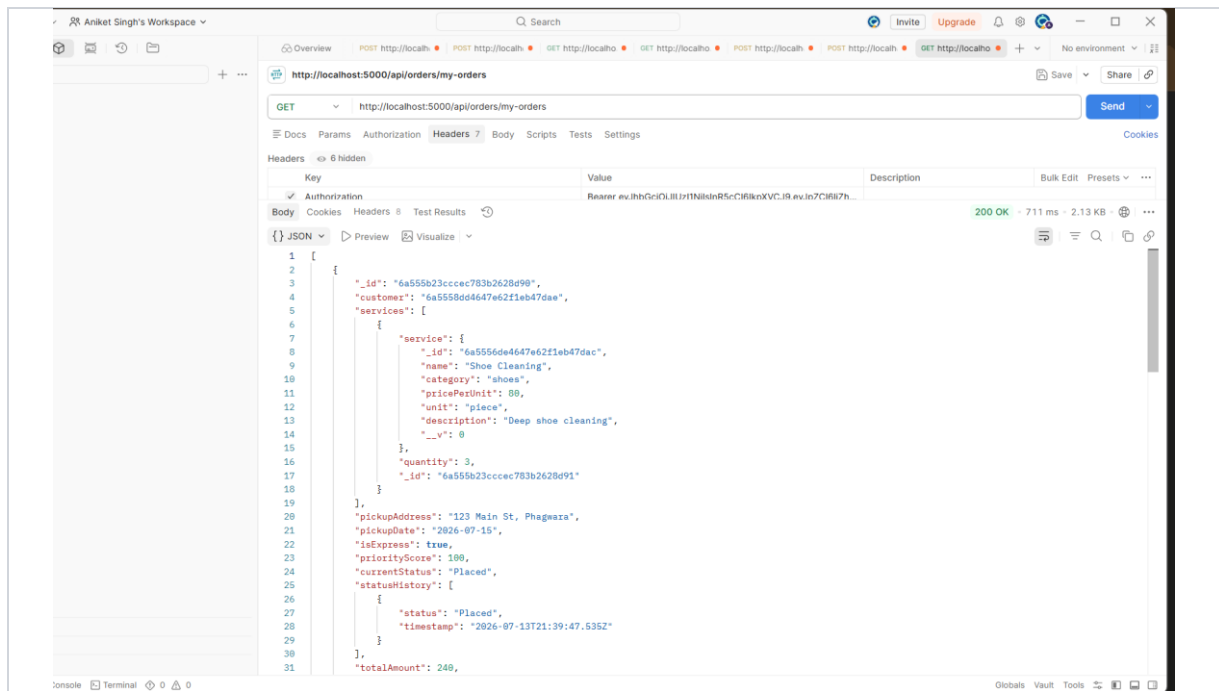


Figure 17: 6.2 Services, Orders, and Priority Queue Tests

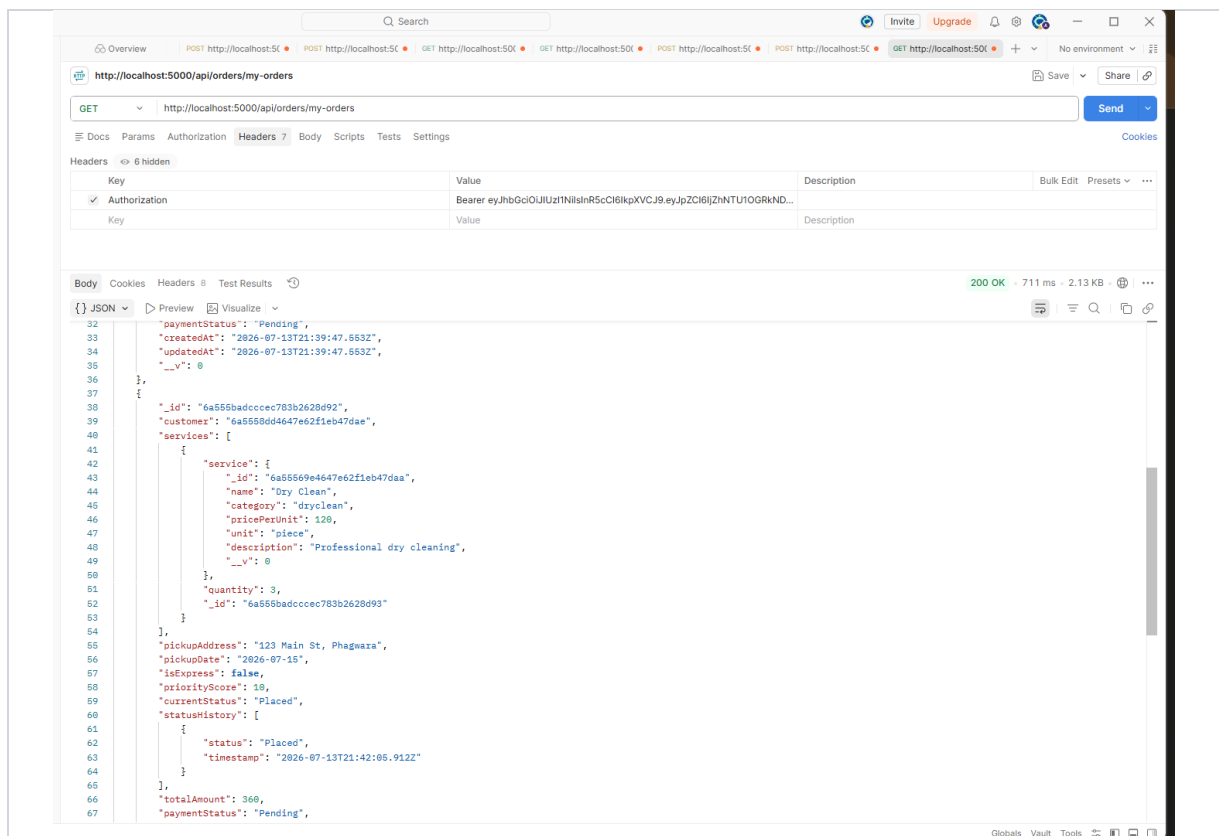


Figure 18: 6.2 Services, Orders, and Priority Queue Tests

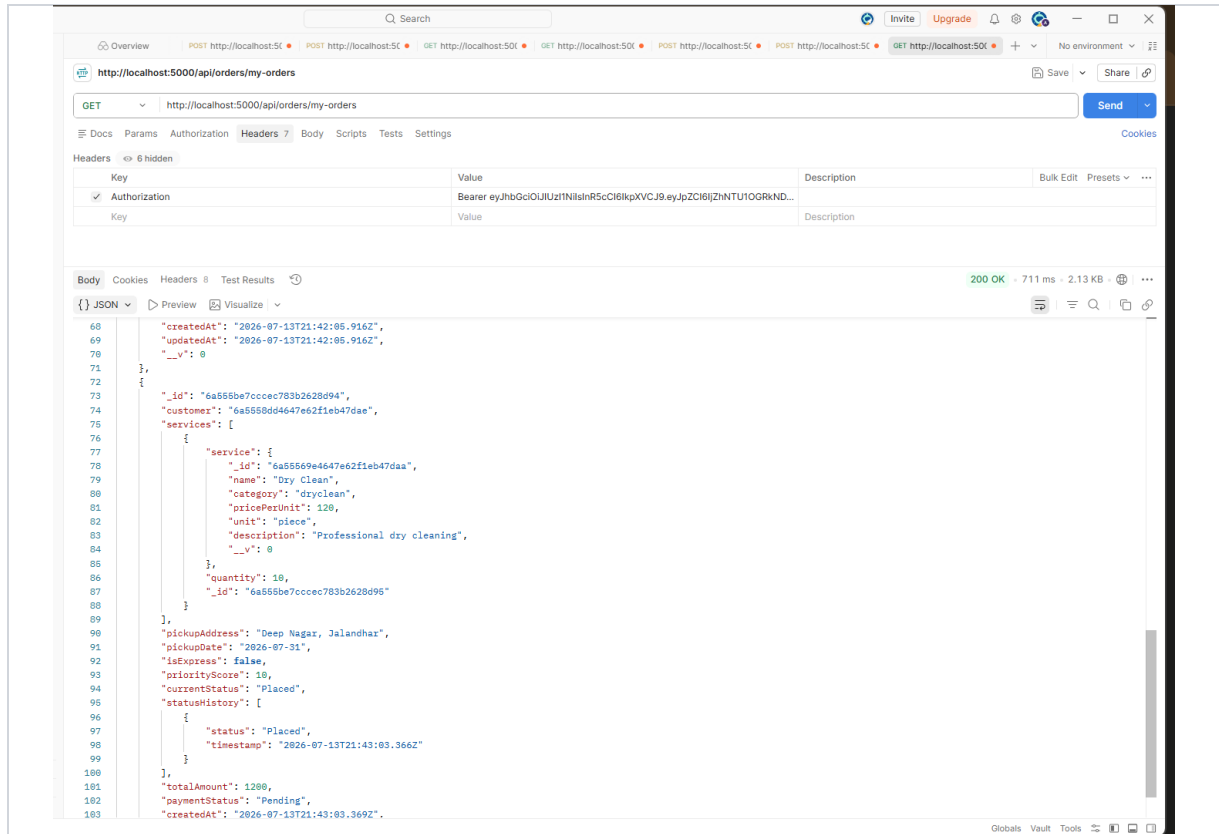


Figure 19: 6.2 Services, Orders, and Priority Queue Tests

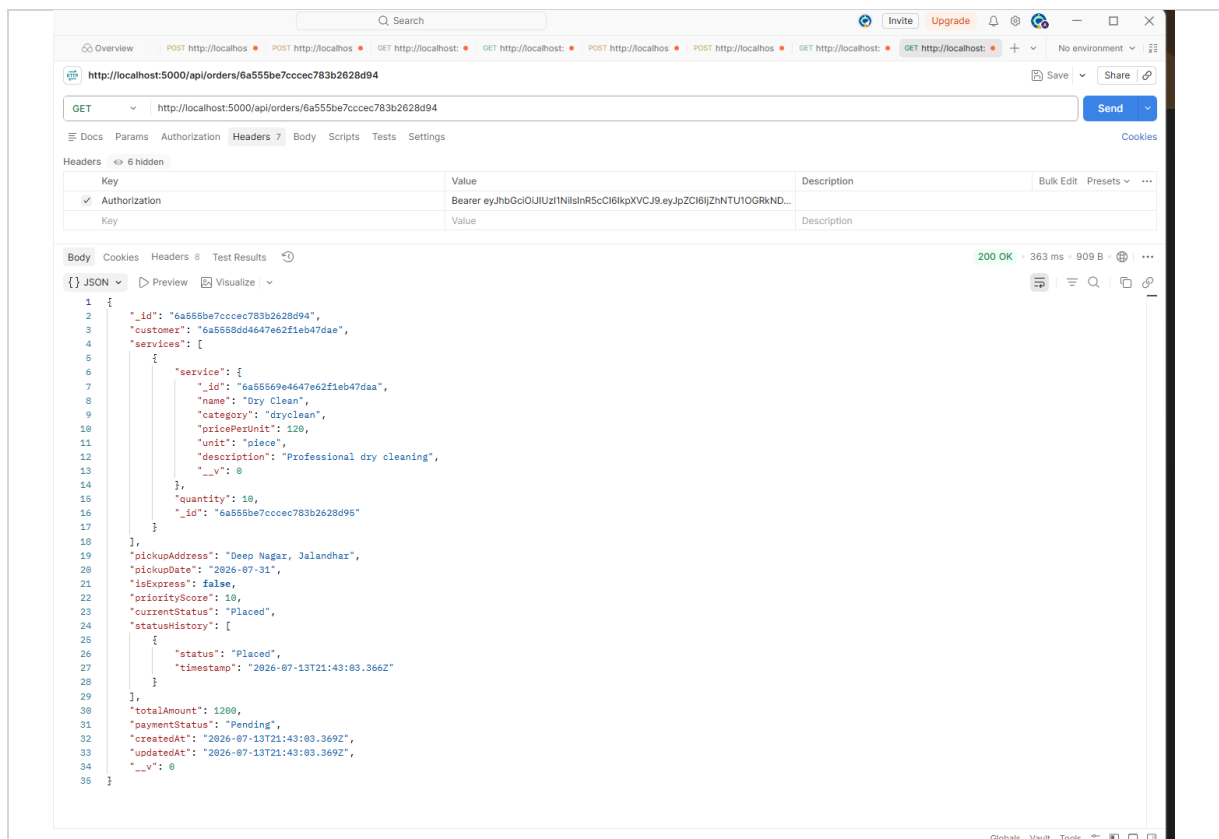


Figure 20: 6.2 Services, Orders, and Priority Queue Tests

6.3 Slot Scheduling, Route Optimization & Frontend Build

Interval-scheduling conflict-detection tests, Dijkstra route-optimization tests, and initial frontend page renders.

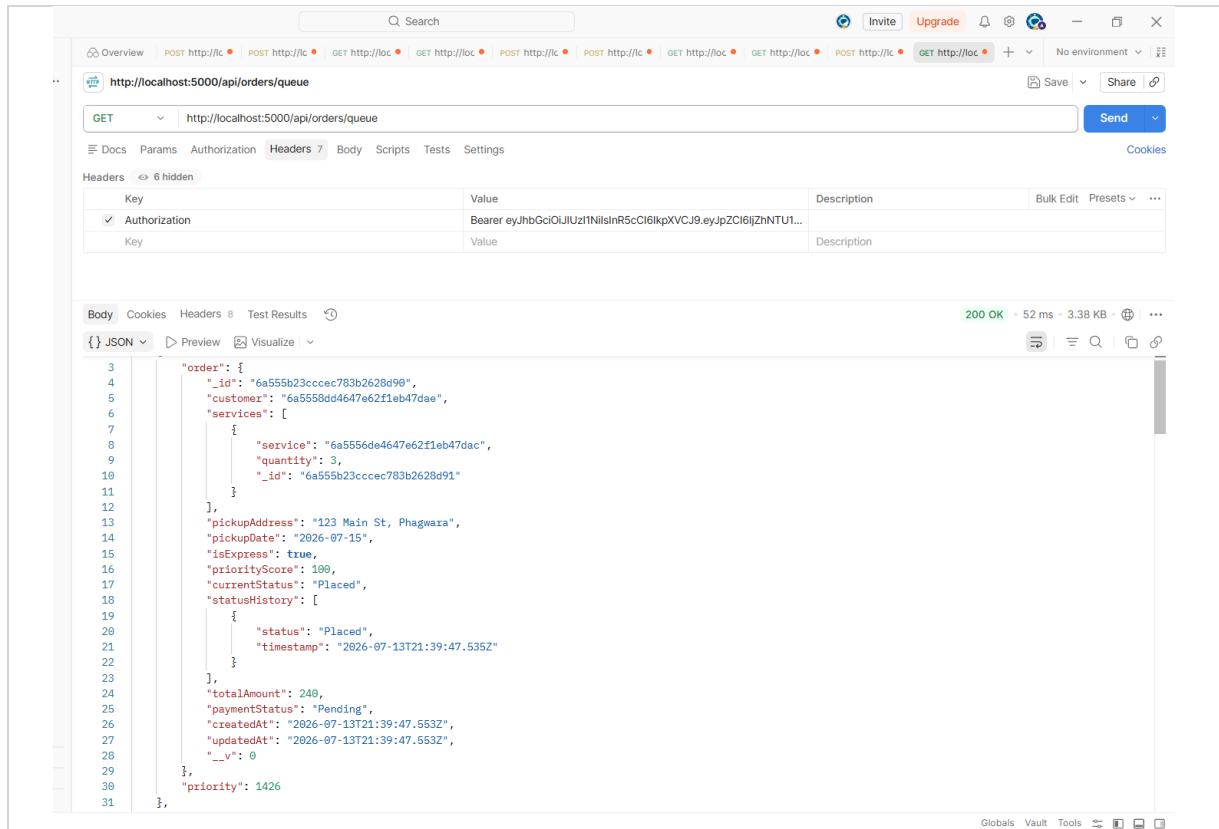


Figure 21: 6.3 Slot Scheduling, Route Optimization & Frontend Build

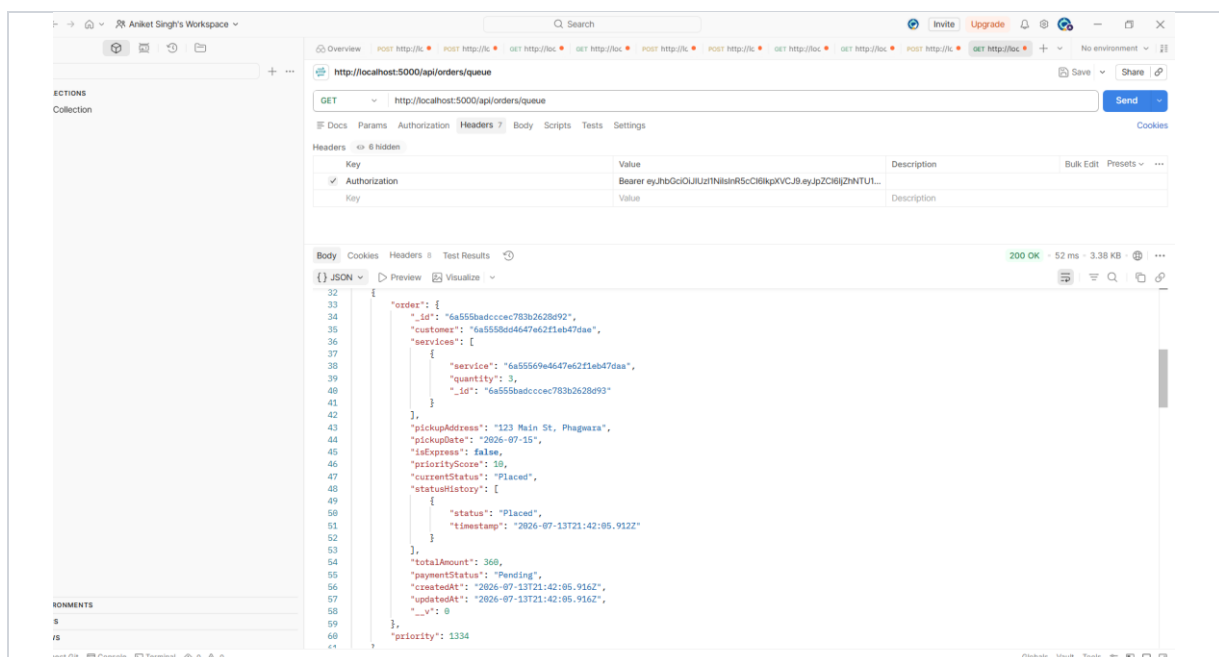


Figure 22: 6.3 Slot Scheduling, Route Optimization & Frontend Build

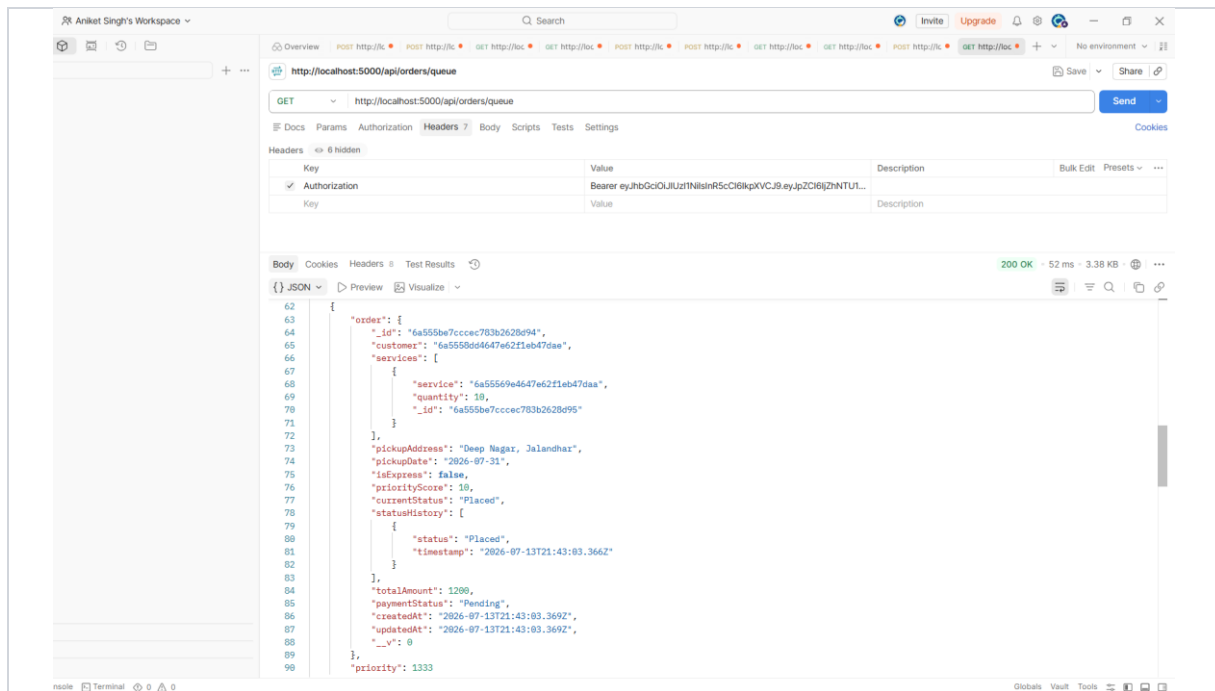


Figure 23: 6.3 Slot Scheduling, Route Optimization & Frontend Build

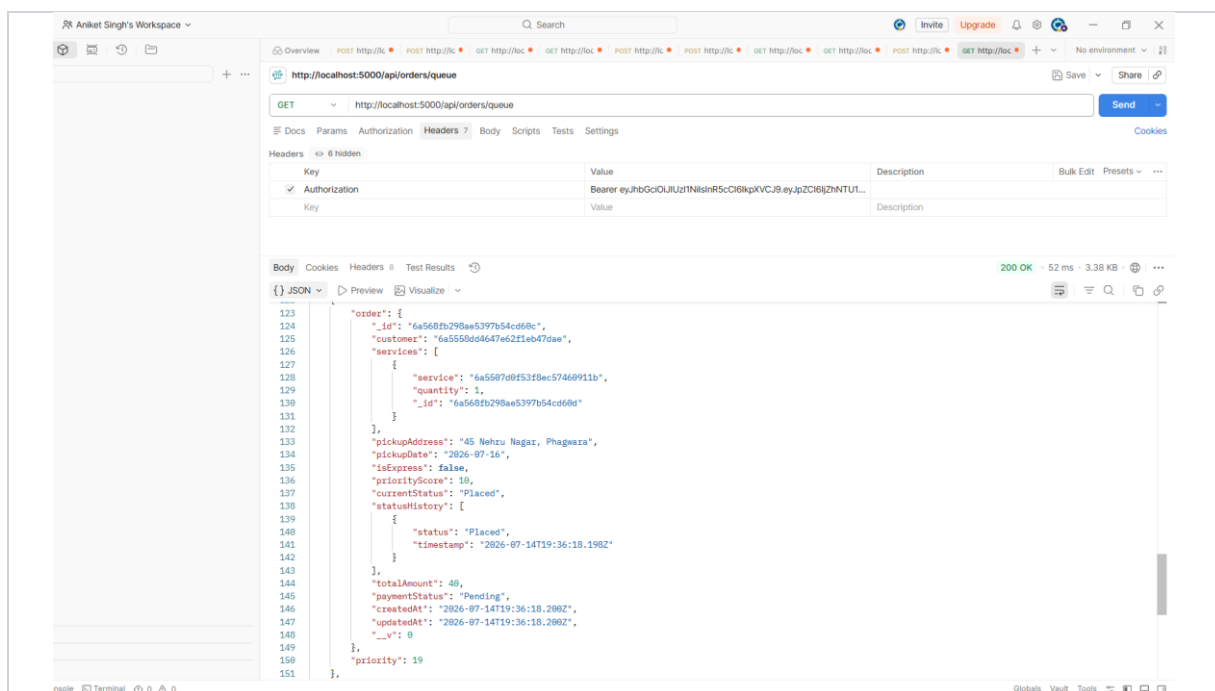


Figure 24: 6.3 Slot Scheduling, Route Optimization & Frontend Build

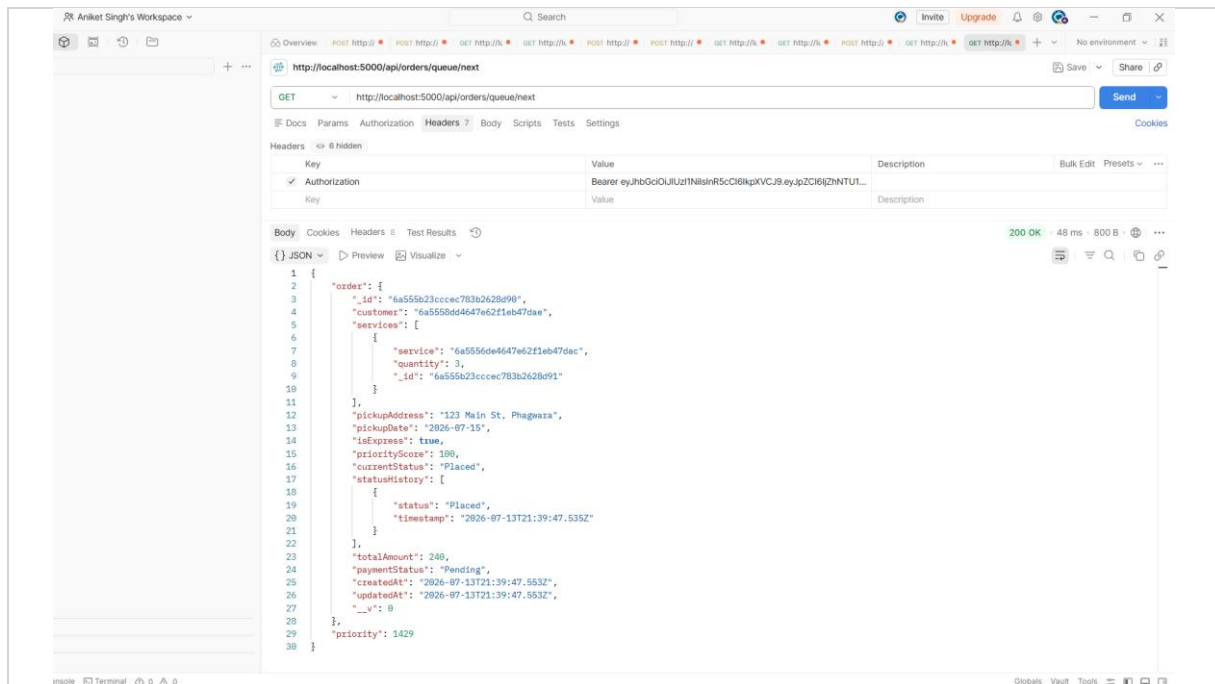


Figure 25: 6.3 Slot Scheduling, Route Optimization & Frontend Build

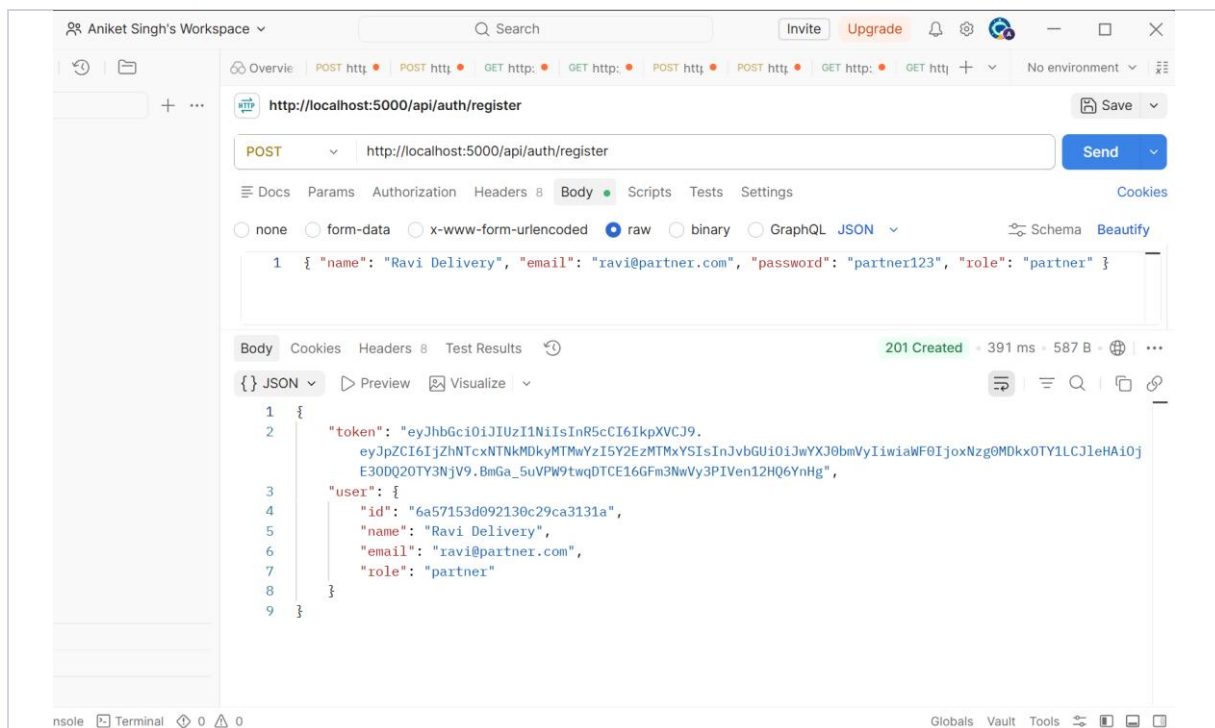


Figure 26: 6.3 Slot Scheduling, Route Optimization & Frontend Build

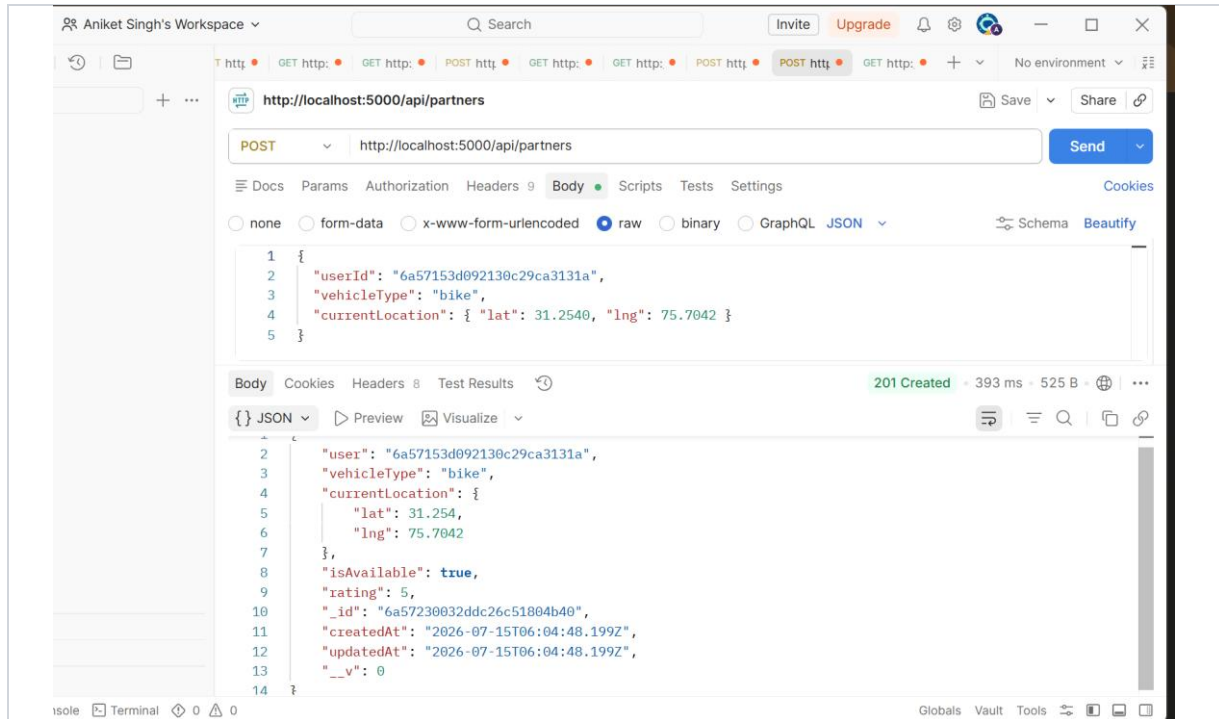


Figure 27: 6.3 Slot Scheduling, Route Optimization & Frontend Build

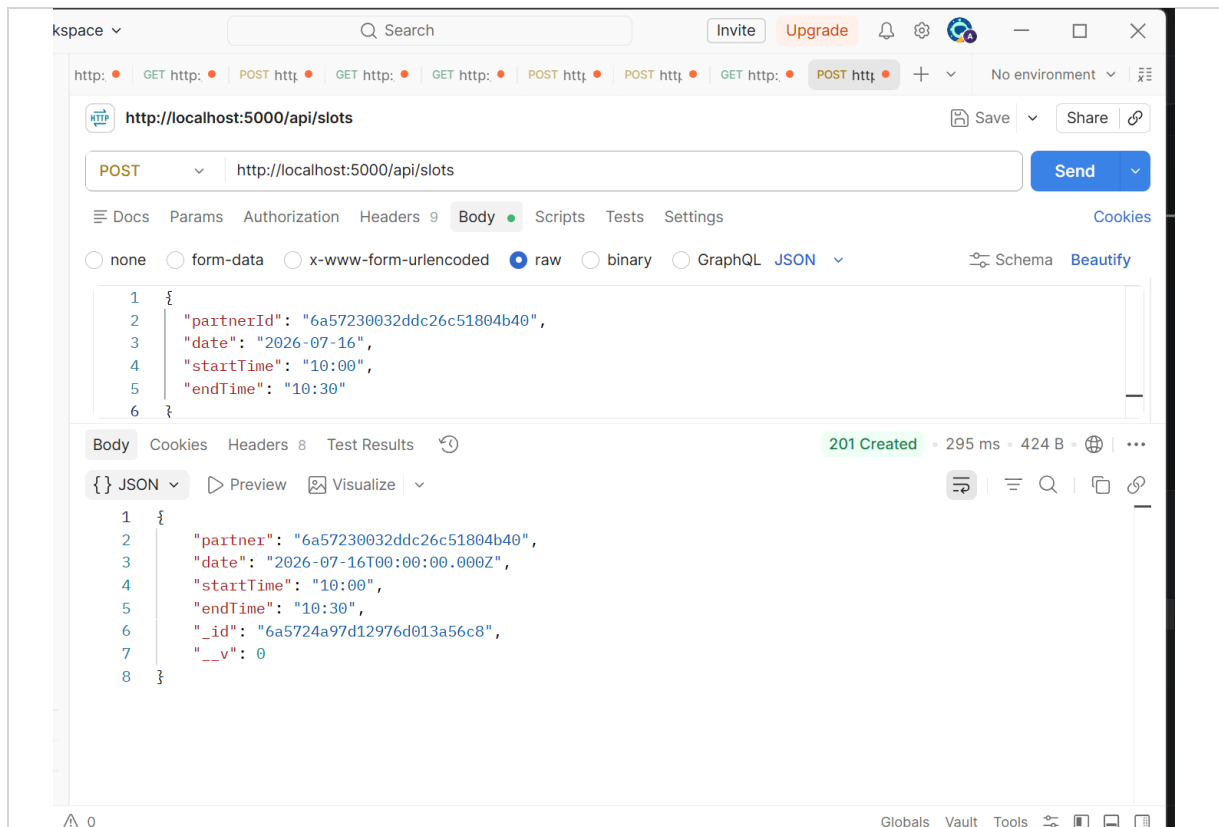


Figure 28: 6.3 Slot Scheduling, Route Optimization & Frontend Build

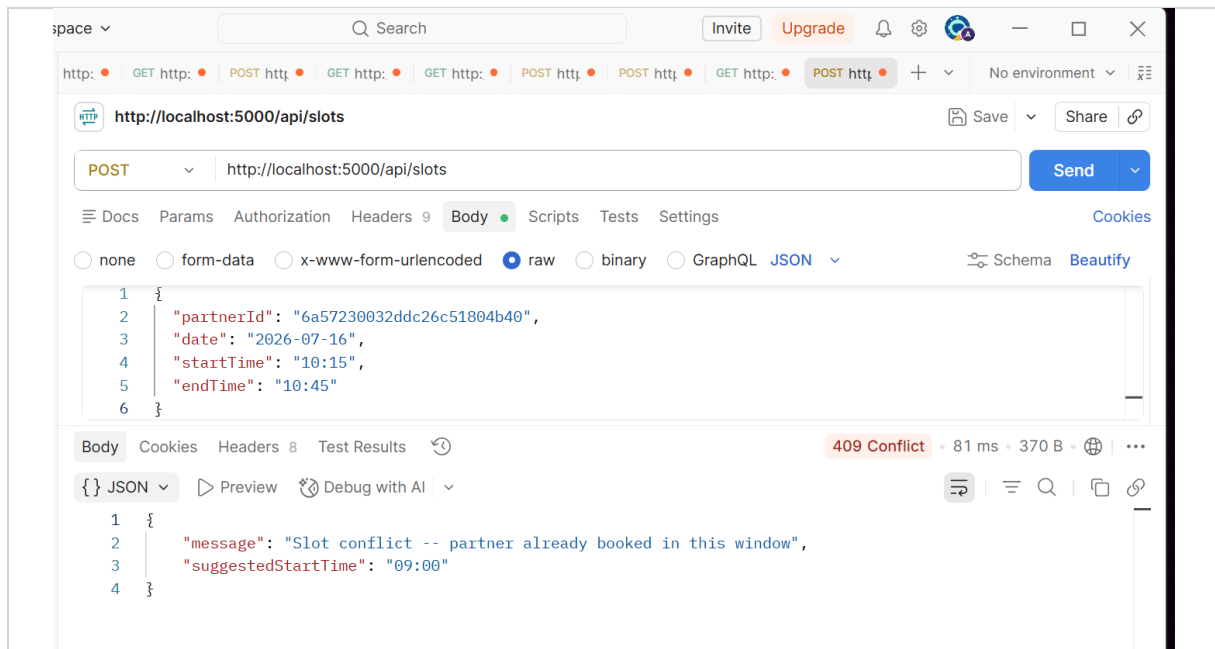


Figure 29: 6.3 Slot Scheduling, Route Optimization & Frontend Build

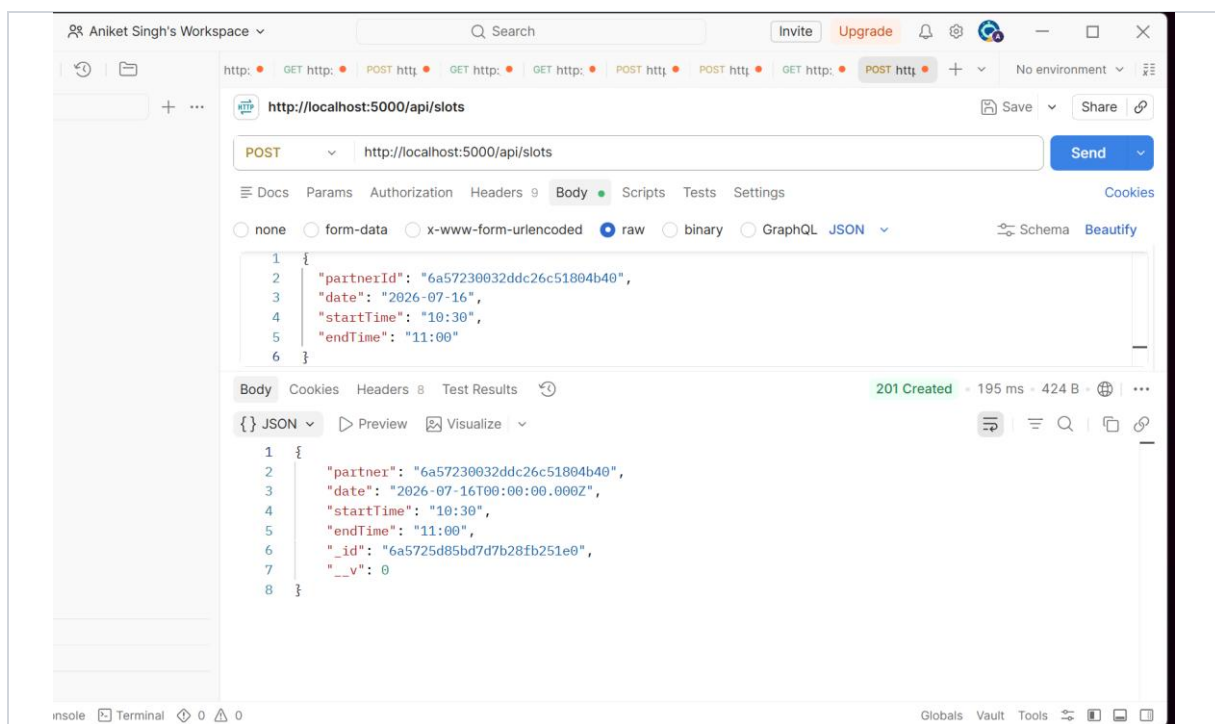


Figure 30: 6.3 Slot Scheduling, Route Optimization & Frontend Build

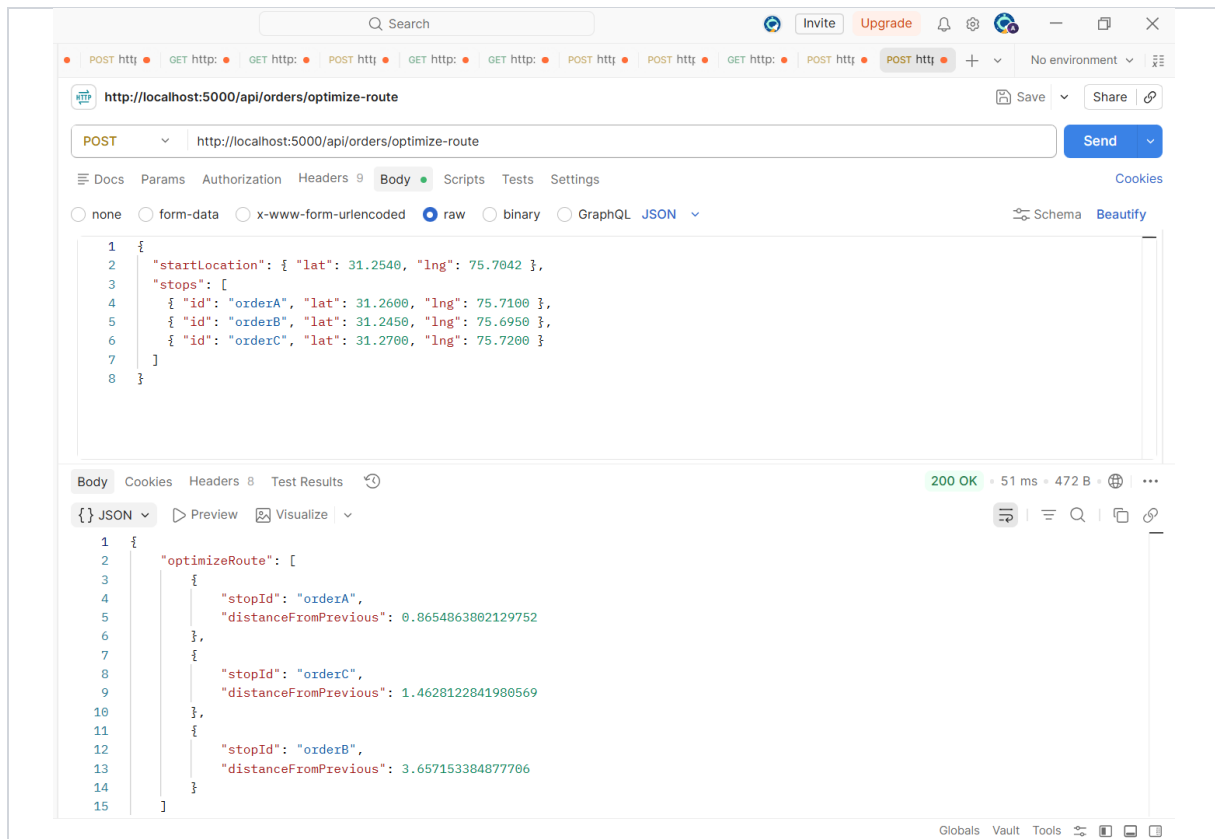


Figure 31: 6.3 Slot Scheduling, Route Optimization & Frontend Build

6.4 Payments, Reviews & Live Socket Tracking

Payment processing tests, invoice generation, review submission, and live Socket.io status-update verification.

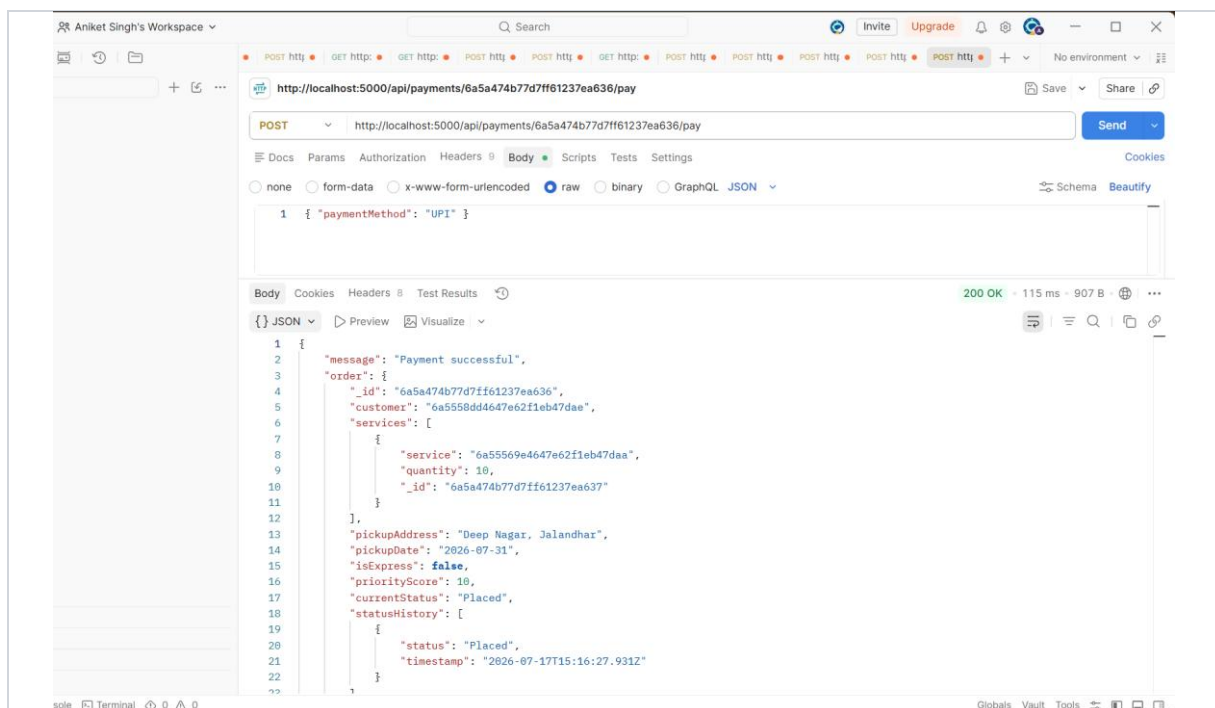


Figure 32: Payments, Reviews & Live Socket Tracking

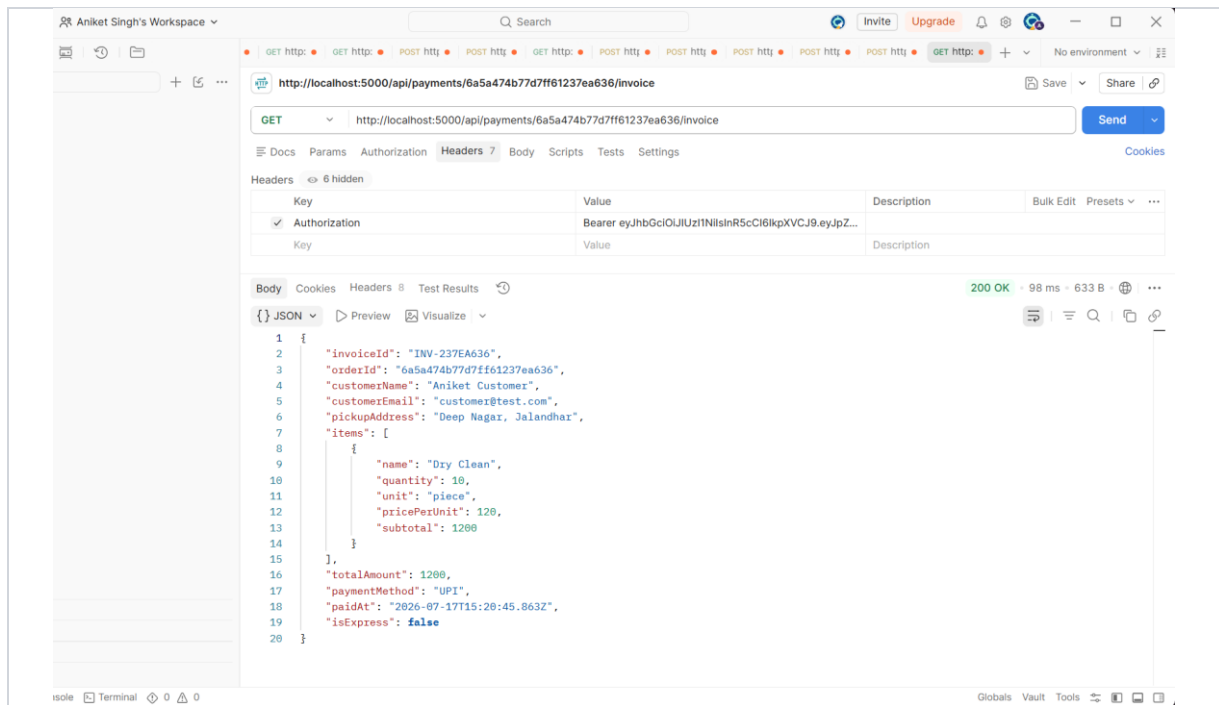


Figure 33: Payments, Reviews & Live Socket Tracking

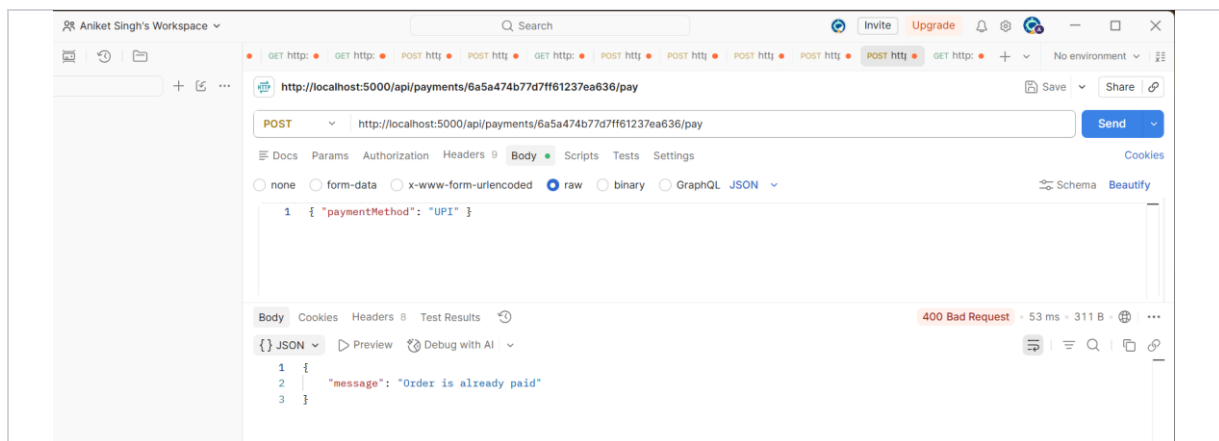


Figure 34: 6.4 Payments, Reviews & Live Socket Tracking

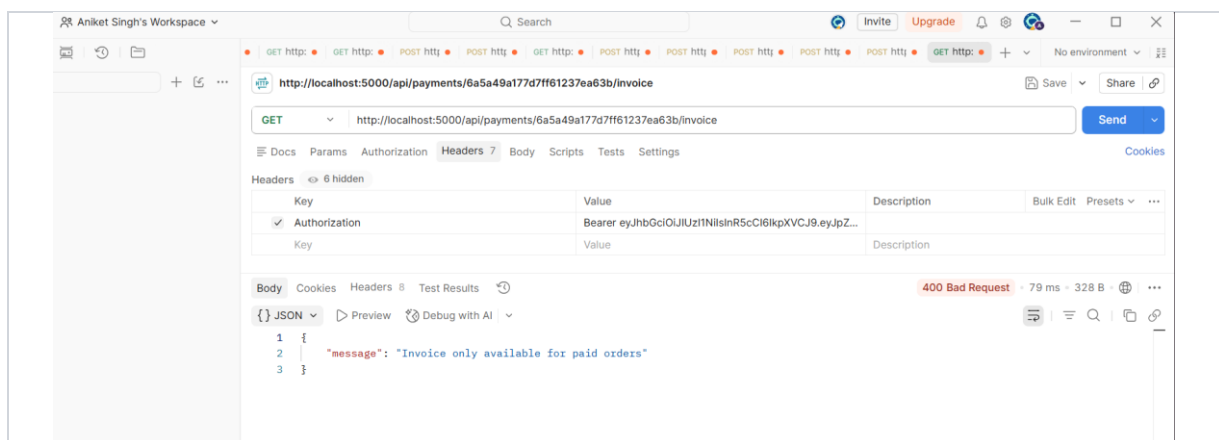


Figure 35: 6.4 Payments, Reviews & Live Socket Tracking

6.5 Bug Fixes, GitHub Upload & Deployment Verification

Final debugging round, GitHub repository setup, Render backend deployment logs, and Vercel frontend deployment confirmation.

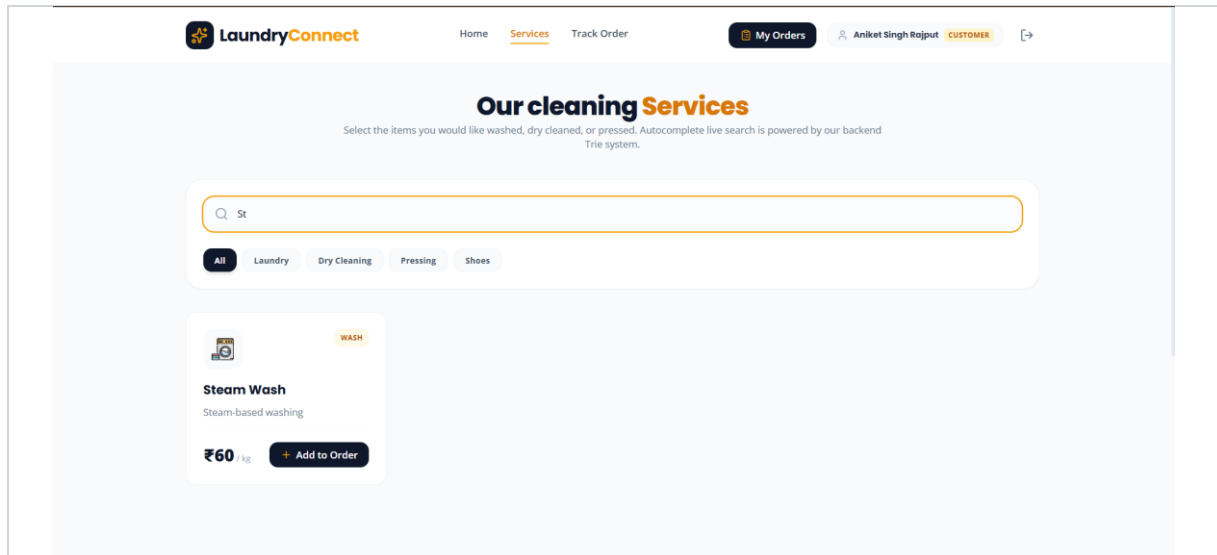


Figure 36: 6.5 Bug Fixes, GitHub Upload & Deployment Verification

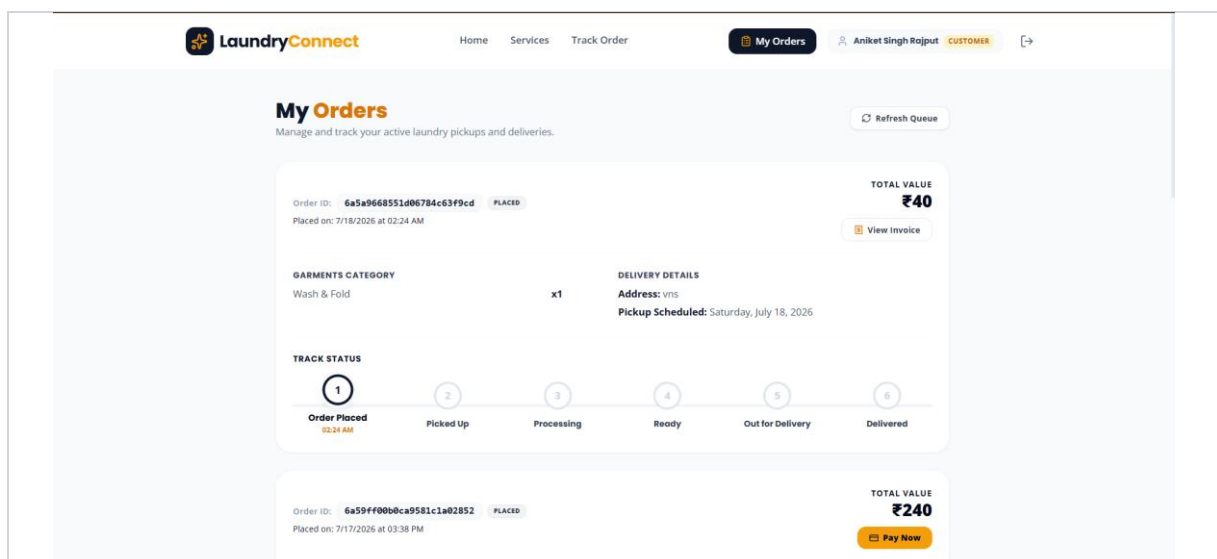


Figure 37: 6.5 Bug Fixes, GitHub Upload & Deployment Verification

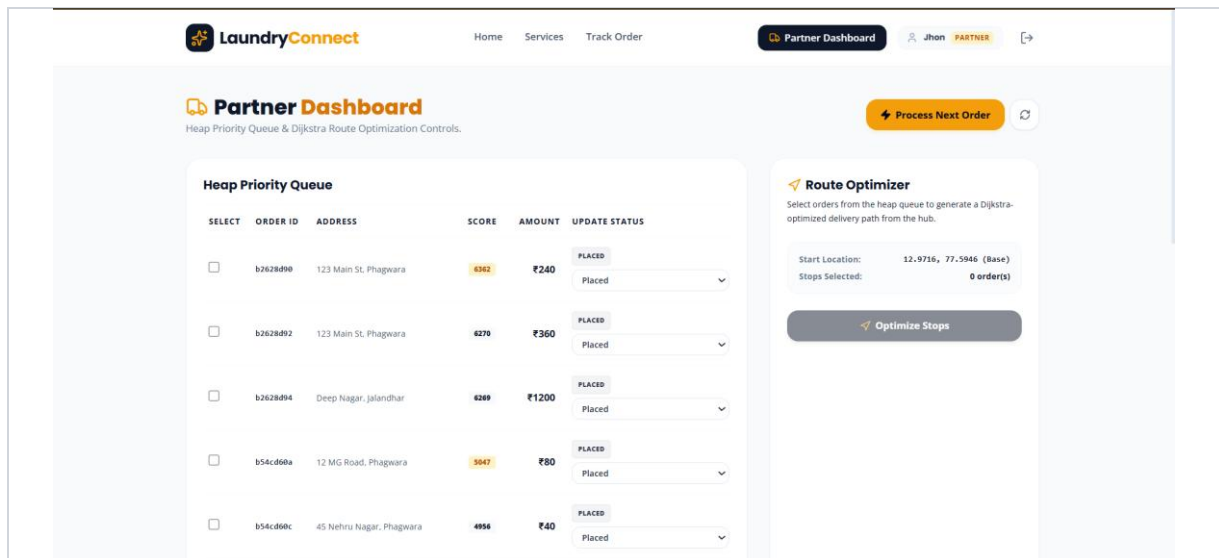


Figure 38: 6.5 Bug Fixes, GitHub Upload & Deployment Verification

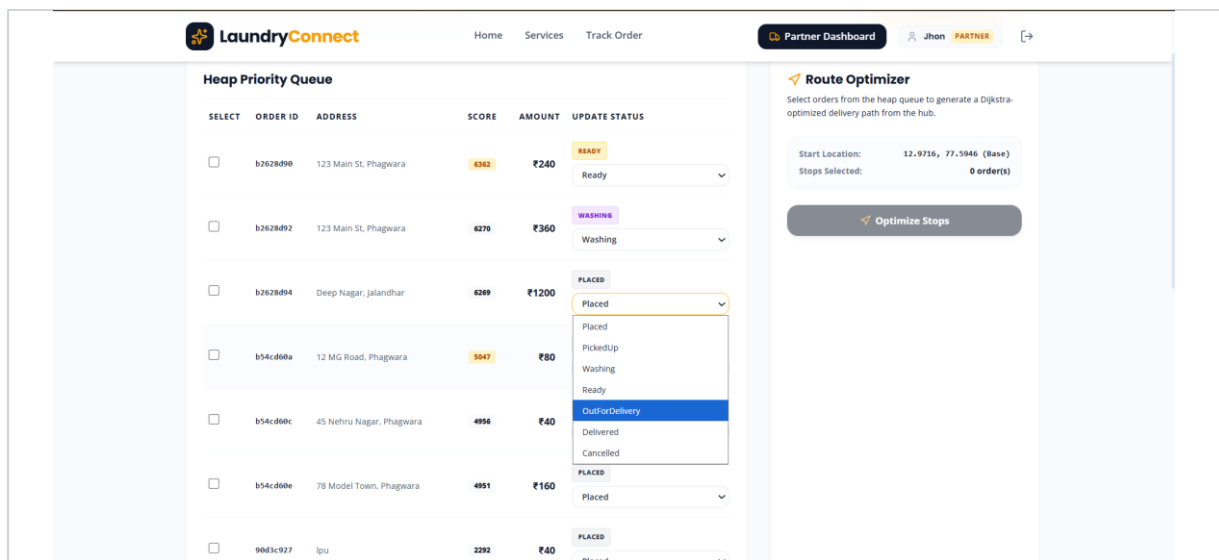


Figure 39: 6.5 Bug Fixes, GitHub Upload & Deployment Verification

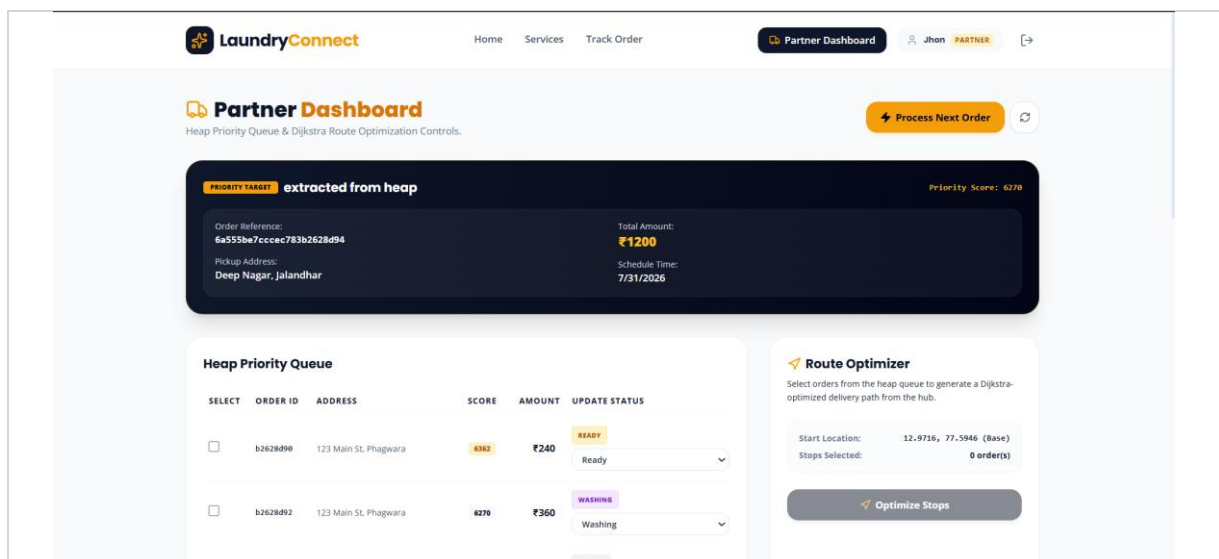


Figure 40: 6.5 Bug Fixes, GitHub Upload & Deployment Verification

LaundryConnect Home Services Track Order Partner Dashboard John PARTNER

Partner Dashboard
Heap Priority Queue & Dijkstra Route Optimization Controls.

Heap Priority Queue

SELECT	ORDER ID	ADDRESS	SCORE	AMOUNT	UPDATE STATUS
☒	b262d94	Deep Nagar, Jalandhar	6285	₹1200	OUTFORDELIVERY OutForDelivery
☒	b54cd60a	12 MG Road, Phagwara	5063	₹80	OUTFORDELIVERY OutForDelivery
☒	b54cd60c	45 Nehru Nagar, Phagwara	4972	₹40	OUTFORDELIVERY OutForDelivery
☒	b54cd60e	78 Model Town, Phagwara	4967	₹160	WASHING Washing
☒	90d3c927	ipu	2308	₹40	DELIVERED Delivered
☒	c1a02845	[PU]	1223	₹95	PICKEDUP

Route Optimizer
Select orders from the heap queue to generate a Dijkstra-optimized delivery path from the hub.

Start Location: 12.9716, 77.5946 (Base)
Stops Selected: 6 order(s)

Optimize Stops

OPTIMIZED PATH STOPS

- Stop 1: Order #b54cd60a
Distance from prev: 0.17 km
- Stop 2: Order #b54cd60c
Distance from prev: 0.39 km
- Stop 3: Order #90d3c927
Distance from prev: 0.88 km
- Stop 4: Order #b262d94
Distance from prev: 0.88 km
- Stop 5: Order #c1a02845
Distance from prev: 0.00 km
- Stop 6: Order #b54cd60e
Distance from prev: 0.46 km

Figure 41: 6.5 Bug Fixes, GitHub Upload & Deployment Verification

LaundryConnect Home Services Track Order My Orders Aniket Singh Rajput CUSTOMER

How It Works
Get fresh clothes back in four easy steps without lifting a finger.

- Schedule**
Select services and choose a convenient pickup date.
- Pickup**
Our partner arrives at your door to bag your items.
- Clean**
Garments are processed using premium, certified care.
- Deliver**
Fresh, folded, pristine clothes are returned to you.

Our Popular Services
Professional cleaning services tailored to keep your garments pristine.

[View All Services →](#)

Figure 42: 6.5 Bug Fixes, GitHub Upload & Deployment Verification

LaundryConnect Home Services Track Order My Orders Aniket Singh Rajput CUSTOMER

Our Popular Services
Professional cleaning services tailored to keep your garments pristine.

[View All Services →](#)

Wash & Fold
Regular wash and fold service
Starting at ₹40/kg

Dry Clean
Professional dry cleaning
Starting at ₹120/piece

Iron & Press
Ironing and pressing
Starting at ₹15/piece

Shoe Cleaning
Deep shoe cleaning
Starting at ₹80/piece

Engineered for Reliability

Figure 43: 6.5 Bug Fixes, GitHub Upload & Deployment Verification

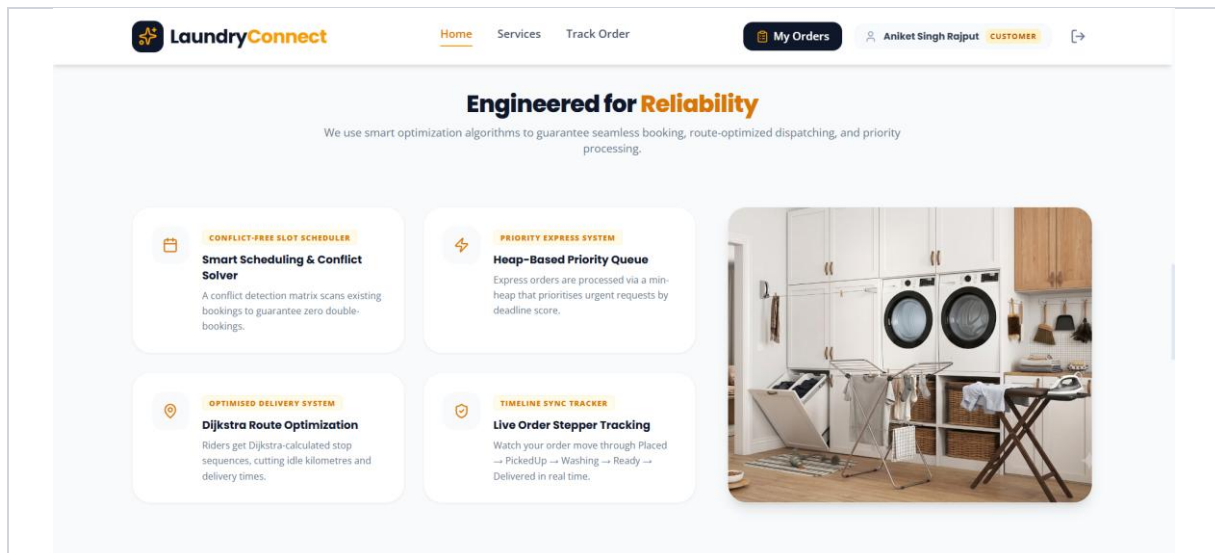


Figure 44: 6.5 Bug Fixes, GitHub Upload & Deployment Verification

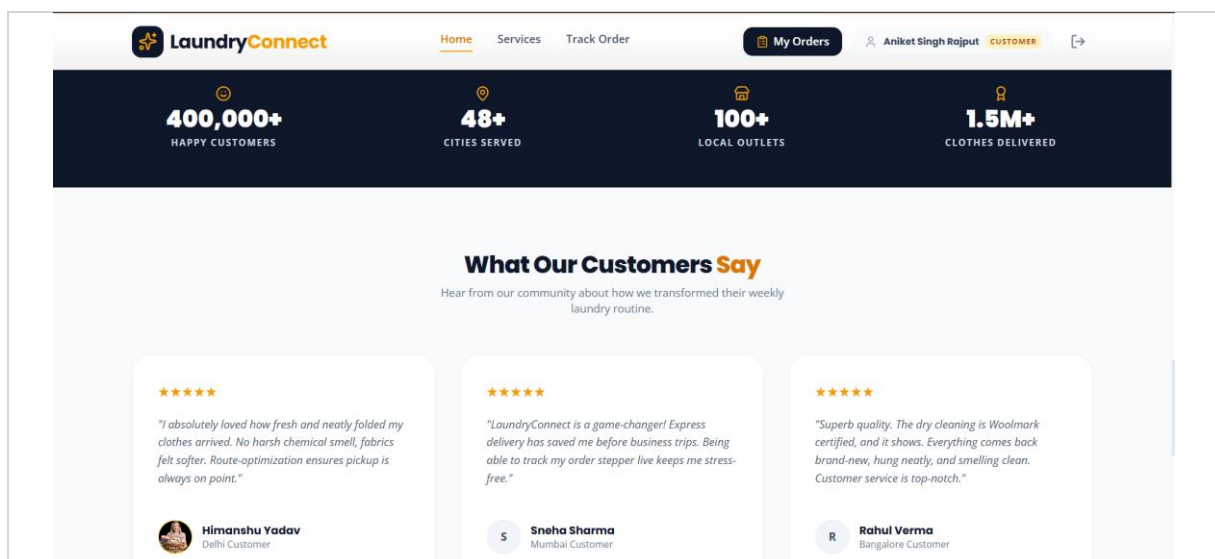


Figure 45: 6.5 Bug Fixes, GitHub Upload & Deployment Verification

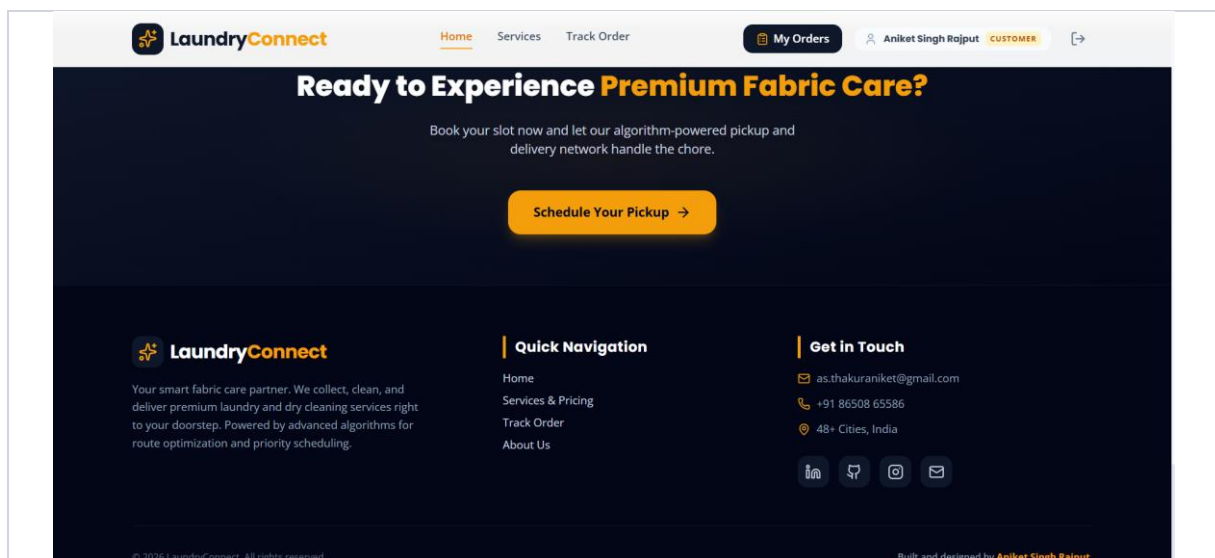
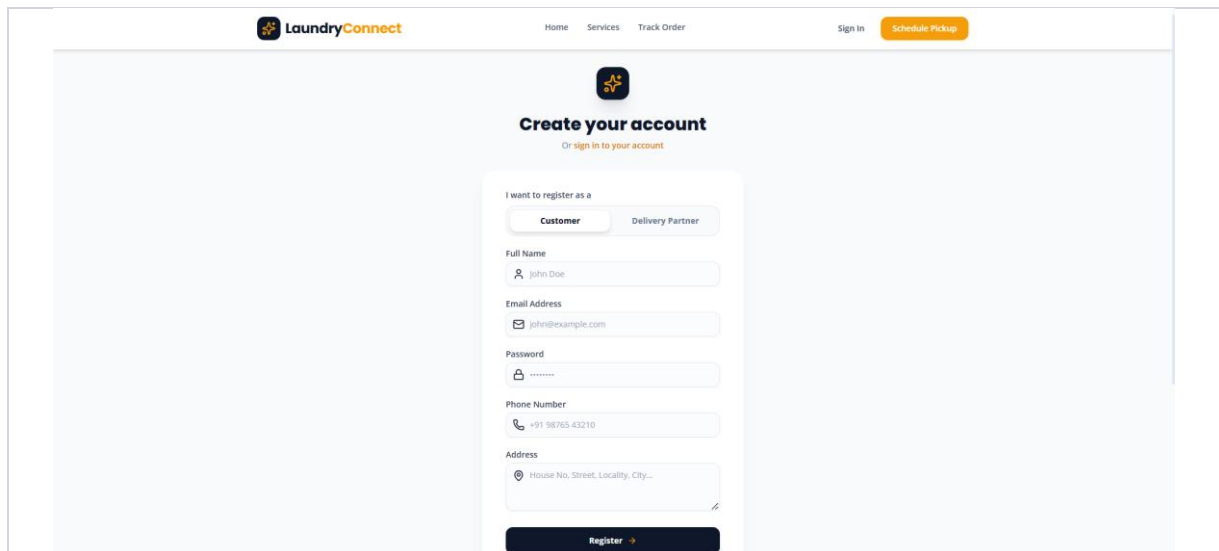
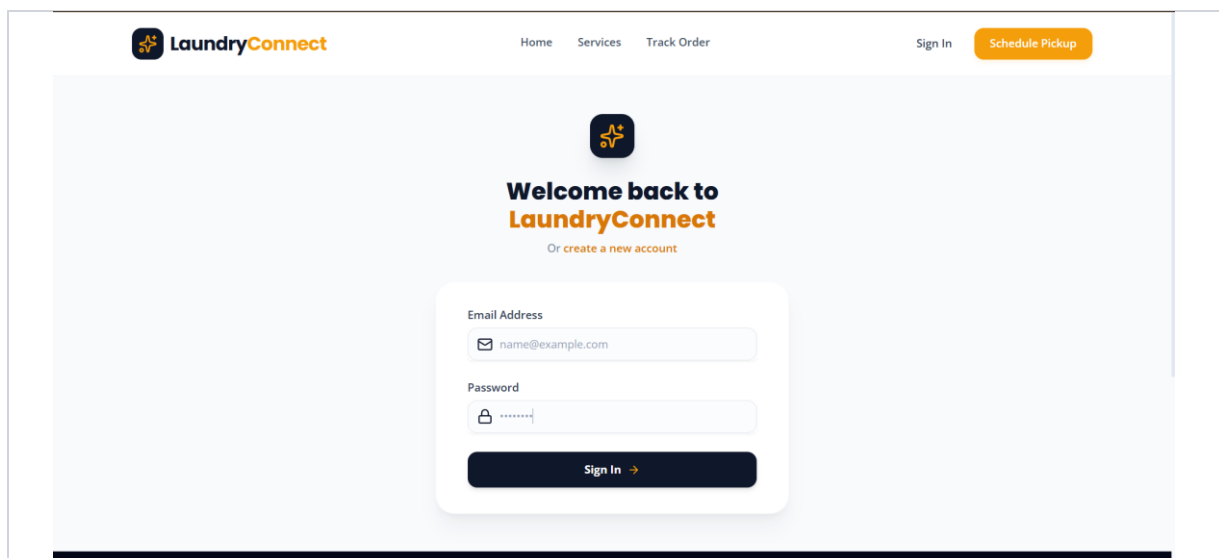


Figure 46: 6.5 Bug Fixes, GitHub Upload & Deployment Verification



The screenshot shows the 'Create your account' page of the LaundryConnect application. The page has a light blue background. At the top, there is a navigation bar with the LaundryConnect logo, links for Home, Services, and Track Order, and buttons for Sign In and Schedule Pickup. The main heading is 'Create your account' with a subtext 'Or sign in to your account'. Below this, there is a form titled 'I want to register as a' with two radio buttons: 'Customer' (selected) and 'Delivery Partner'. The form includes input fields for Full Name (John Doe), Email Address (john@example.com), Password (masked with dots), Phone Number (+91 98765 43210), and Address (House No, Street, Locality, City...). A 'Register' button is at the bottom of the form.

Figure 47: 6.5 Bug Fixes, GitHub Upload & Deployment Verification



The screenshot shows the 'Welcome back to LaundryConnect' page of the application. The page has a light blue background. At the top, there is a navigation bar with the LaundryConnect logo, links for Home, Services, and Track Order, and buttons for Sign In and Schedule Pickup. The main heading is 'Welcome back to LaundryConnect' with a subtext 'Or create a new account'. Below this, there is a form with input fields for Email Address (name@example.com) and Password (masked with dots). A 'Sign In' button is at the bottom of the form.

Figure 48: 6.5 Bug Fixes, GitHub Upload & Deployment Verification

CHAPTER 7: USER INTERFACE SCREENSHOTS

The following screenshots showcase the final, deployed user interface across the core customer and administrative workflows.

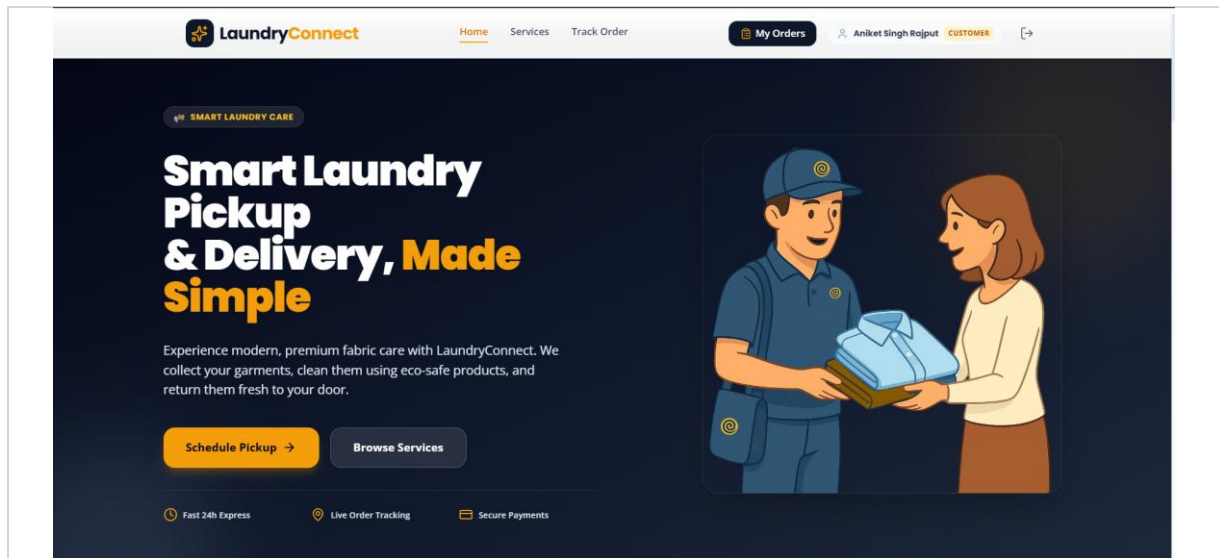


Figure 49: Home page — hero section with live delivery illustration and quick-action buttons

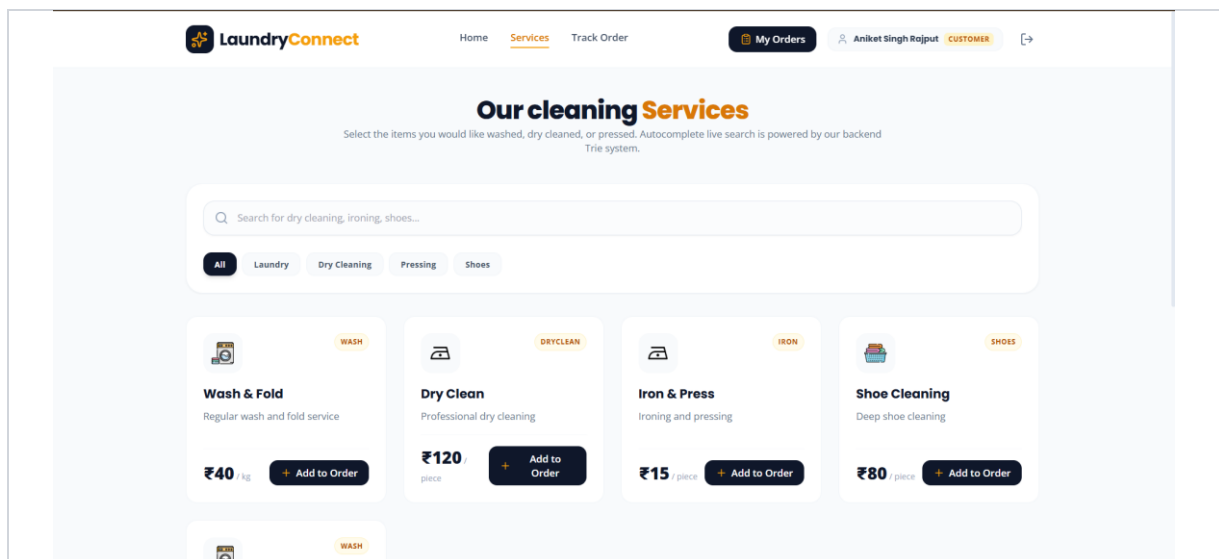


Figure 50: Services page — Trie-powered live search and category filtering

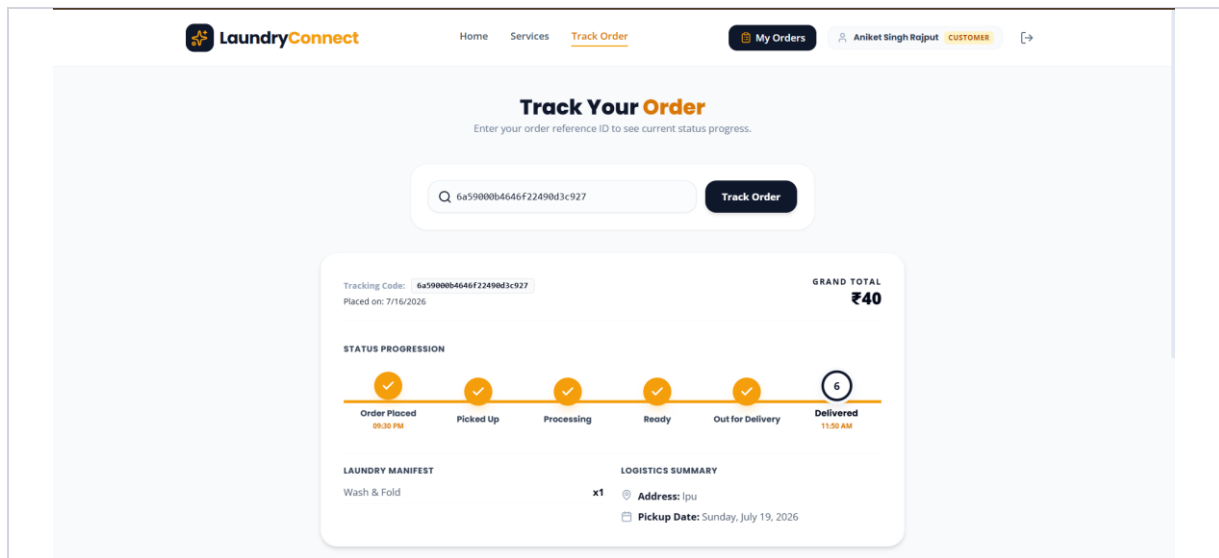


Figure 51: Track Order page — real-time status timeline updated via Socket.io

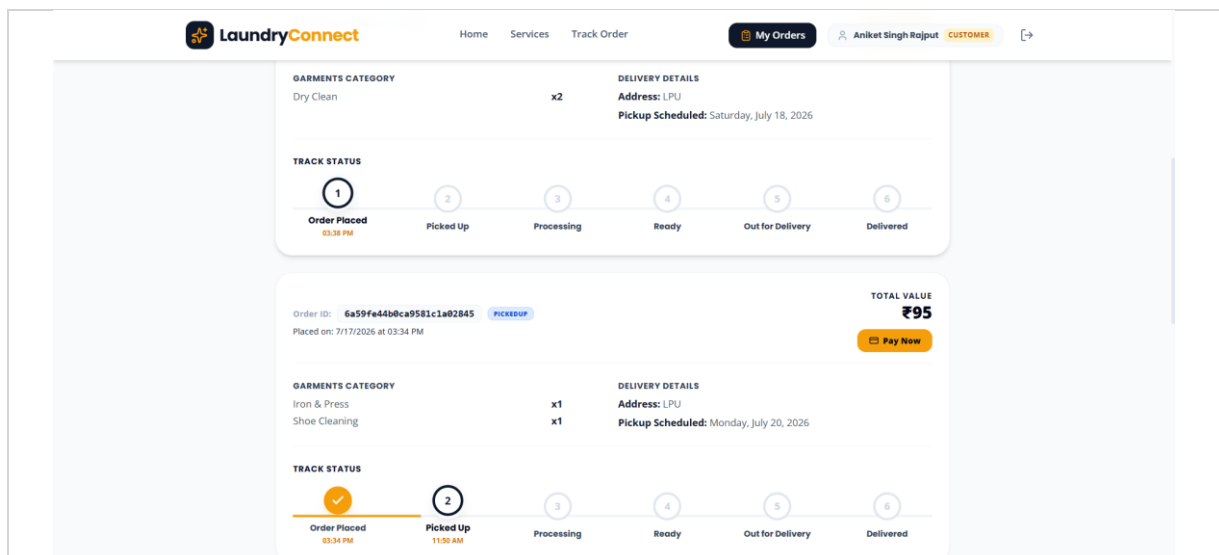


Figure 52: Admin Dashboard — priority queue, partner management, and service catalogue

CHAPTER 8: CHALLENGES FACED AND SOLUTIONS

Building and integrating a full-stack application within one week surfaced several real, instructive bugs. Diagnosing and resolving each strengthened understanding of the full request/response and data-flow lifecycle across the stack.

Challenge	Root Cause	Solution
Generic-looking UI despite functional backend	Service icons rendered as emoji characters instead of real image assets	Replaced every emoji icon with mapped, category-specific image assets
Hero image displayed broken	Hardcoded external stock-photo URL with no local fallback	Moved all imagery to local /public assets, removed external dependencies entirely
Submitted review text never displayed	Field-name mismatch: frontend read/wrote “review” while the schema stored “reviewComment”	Traced the schema-to-component data flow and corrected both the read and write sites
Optimized delivery route never rendered despite a successful 200 response	One-character key mismatch between the API response (“optimizeRoute”) and frontend state setter (“optimizedRoute”)	Standardized on the grammatically correct “optimizedRoute” key on the backend, matching the rest of the API’s naming convention
“Schedule Pickup” redirected logged-in users to the Register page	Misconfigured protected-route guard / stale auth-state check	Audited ProtectedRoute logic and route definitions; corrected the redirect target
Category filter tabs returned “No Services Found” for every tab	Filter tab labels did not match the actual lowercase category values stored in MongoDB	Rebuilt the label-to-category mapping to match the real seeded data
Initial GitHub push rejected (“unrelated histories”)	GitHub auto-generated a README/license on repo creation, conflicting with the local commit	Recreated the repository with no auto-generated files, ensuring a clean initial push

CHAPTER 9: CONCLUSION & FUTURE SCOPE

9.1 Conclusion

LaundryConnect successfully demonstrates that core Data Structures and Algorithms concepts — typically taught in isolation — can be meaningfully applied to solve real operational problems in a production-style full-stack application. The Trie enables scalable search, the priority queue ensures fair order processing, interval scheduling prevents real scheduling conflicts, and Dijkstra's algorithm produces genuinely optimized delivery routes. Combined with a complete authentication system, real-time WebSocket tracking, a mock payment and invoicing flow, and a fully responsive React frontend, the project stands as both a valid academic DSA evaluation piece and a functionally complete, publicly deployed product.

9.2 Future Enhancements

- Integration with a real payment gateway (Razorpay/Stripe) in place of the current mocked flow.
- A Redis-backed persistent priority queue to support horizontal scaling across multiple server instances.
- Push notifications (web push / SMS) for order-status changes.
- An analytics dashboard for administrators covering revenue trends and partner performance.
- Automated partner-to-order assignment based on live partner location combined with heap priority.
- Server-side PDF invoice generation in place of the current browser print-to-PDF approach.

BIBLIOGRAPHY

- MongoDB, Inc. — MongoDB Atlas Documentation. <https://www.mongodb.com/docs/atlas/>
- Express.js — Official Documentation. <https://expressjs.com/>
- React — Official Documentation. <https://react.dev/>
- Socket.io — Official Documentation. <https://socket.io/docs/>
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. — Introduction to Algorithms (3rd Edition), MIT Press — reference for Heap, Dijkstra's Algorithm, and Interval Scheduling implementations.
- JSON Web Tokens (JWT) — Official Introduction. <https://jwt.io/introduction>
- Tailwind CSS — Official Documentation. <https://tailwindcss.com/docs>
- Render — Deployment Documentation. <https://render.com/docs>
- Vercel — Deployment Documentation. <https://vercel.com/docs>